Description of the project **2**
Part 1: S3 & CloudFront (Global Content Delivery) **5**
Created S3 bucket with an index.html file 5
Created a cloudfront distribution 6
My S3 bucket cannot be accessed by the internet by defauly 7
I add index.html as default root object 8
However, cloudfront distribution is still not accessible 9
I force to have text/html as content-type header with a cloudfront function 10
I invalidate/clear cache in order to take new version of index.html 12
Now my index.html is accessible via cloudfront 13
Part 2: Amazon EFS (Shared Regional Storage) **14**
I create a VPC with two Availability Zones (east-1a and east-1b) 14
I create Security Group that will be attached to the EC2 16
Created first EC2 instance with Public subnet us-east-1a Security Group: prod-ec2-sg 18
Created second EC2 with: Public subnet in us-east-1b Security Group prod-ec2-sg 19
Open the Security Group to inbound SSH connections 20
Sudo mount is timing out 21
I open port 2049 on the Elastic file system Security Group only if inbound connection comes from EC2 Security Group (Security Group Nesting or Chaining) 22
Now EFS mount works correctly: Size is 8GB but but scalable to virtually endless size 23
I am able to copy data in the EFS 24
Now I can see the test.txt in my EFS in AWS with the 12kb 25
Part 3: Storage Gateway (Architectural Knowledge) **30**
I create an S3 Bucket 30
Create large EC2 instance that will simulate our “office storage server” 31
Successfully activated the storage gateway 33
Completed the creation of the storage gateway which now running 34
Create File Share. This allows me to build a bridge between my local computer and the storage. 35
I make sure NFC TCP port is open to 0.0.0.0/0 on the EC2 machine 36
I try and I cannot mount the storage as NFS on my local laptop: 37
I open all the inbound rules of my EC2 machine 38
After 5 minutes I can see the file in the S3 Bucket! 40

Description of the project

This lab focuses on two core architectural patterns: **Global Content Delivery** and **Shared Regional Storage**.

Part 1: S3 & CloudFront (Global Content Delivery)

In this section, you will practice delivering content globally while maintaining a "Zero Trust" security posture for your storage.

- **Step 1: Create a Private S3 Bucket**
  - Create a new S3 bucket (e.g., `my-static-content-12345`).
  - **Crucial:** Keep "Block all public access" **enabled**. this content should not be accessible directly via an S3 URL.
  - Upload an `index.html` file and an image.
- **Step 2: Deploy a CloudFront Distribution**
  - Set the **Origin Domain** to your S3 bucket.
  - **Origin Access:** Select **Origin Access Control (OAC)** (the current AWS best practice).
  - Click "Create Control Setting" and let AWS generate the policy.
  - **Important:** Once the distribution is created, copy the generated **S3 Bucket Policy** and paste it into your S3 bucket's "Permissions" tab. This allows *only* CloudFront to read your files.
- **Step 3: Test and Optimize**
  - Try to access the S3 URL (it should be 403 Forbidden).
  - Access the CloudFront Domain Name (e.g., `d111111abcdef8.cloudfront.net`) (it should load your site).
  - **Edge Logic:** Create a **CloudFront Function** to append a security header (like `X-Content-Type-Options: nosniff`) to all responses. This simulates the "performance optimization" and "security at the edge"

Part 2: Amazon EFS (Shared Regional Storage)

Many legacy or CMS applications (like WordPress) require a shared **file system**.

- **Spin up a clean, simple network:** Don't waste time manual-building a massive 3-tier VPC. Use the "VPC and more" wizard in the AWS console to generate a standard public/private VPC in 60 seconds.
- **Focus on the Security Group configuration:** The SAA exam frequently asks about EFS security. You *must* understand that the EFS Mount Target needs an inbound rule allowing **NFS (Port 2049)** where the source is the **Security Group of the EC2 instances**. (AWS loves testing security group nesting).
- **Test the "Shared State":** Once you mount EFS on two different EC2 instances, write a file on one and watch it appear on the other. This cements the concept of "concurrent file access" which distinguishes EFS from EBS (which is usually single-instance block storage).

Part 3: Storage Gateway (Architectural Knowledge)

Since Storage Gateway requires an on-premises VM, you can practice this via a "Thought Lab" or by using the AWS Management Console to walk through the setup:

Provision the Host (The "Office Server"): Launch an `m5.xlarge` EC2 instance using the official AWS Storage Gateway image. Attach a secondary **150 GiB EBS volume** to serve strictly as the local data cache.

Network Handshake: Open **Port 80** (for initial activation setup) and **Port 2049** (NFS storage traffic) on the instance's security group so your computer can talk to it.

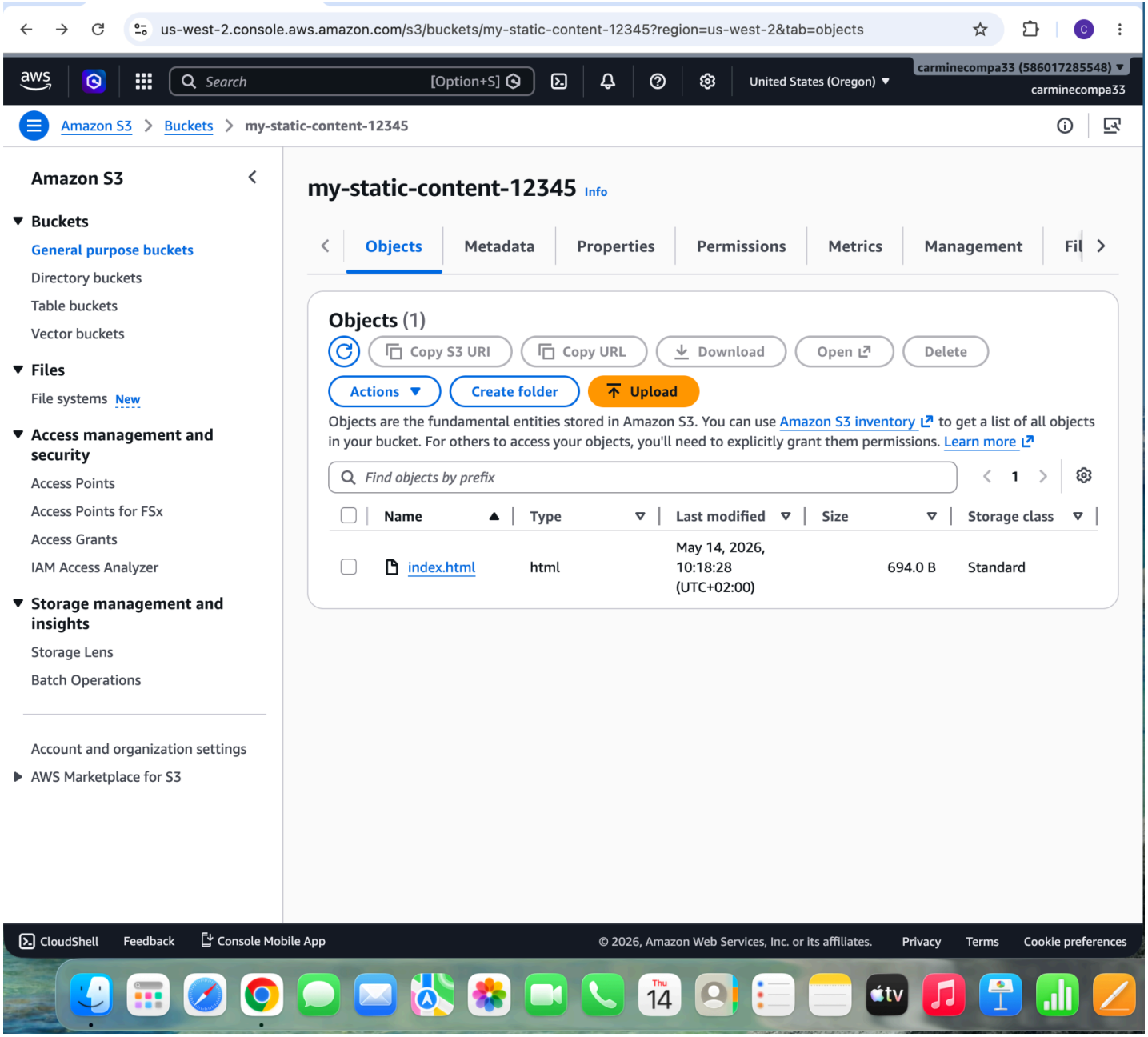
Gateway Activation: Link the running EC2 instance to your AWS Storage Gateway account console by entering its public IP address, then format that extra 150 GiB disk as the **Cache**.

Create the File Share: Map your newly activated gateway to a target private **S3 Bucket** via the NFS file sharing protocol.

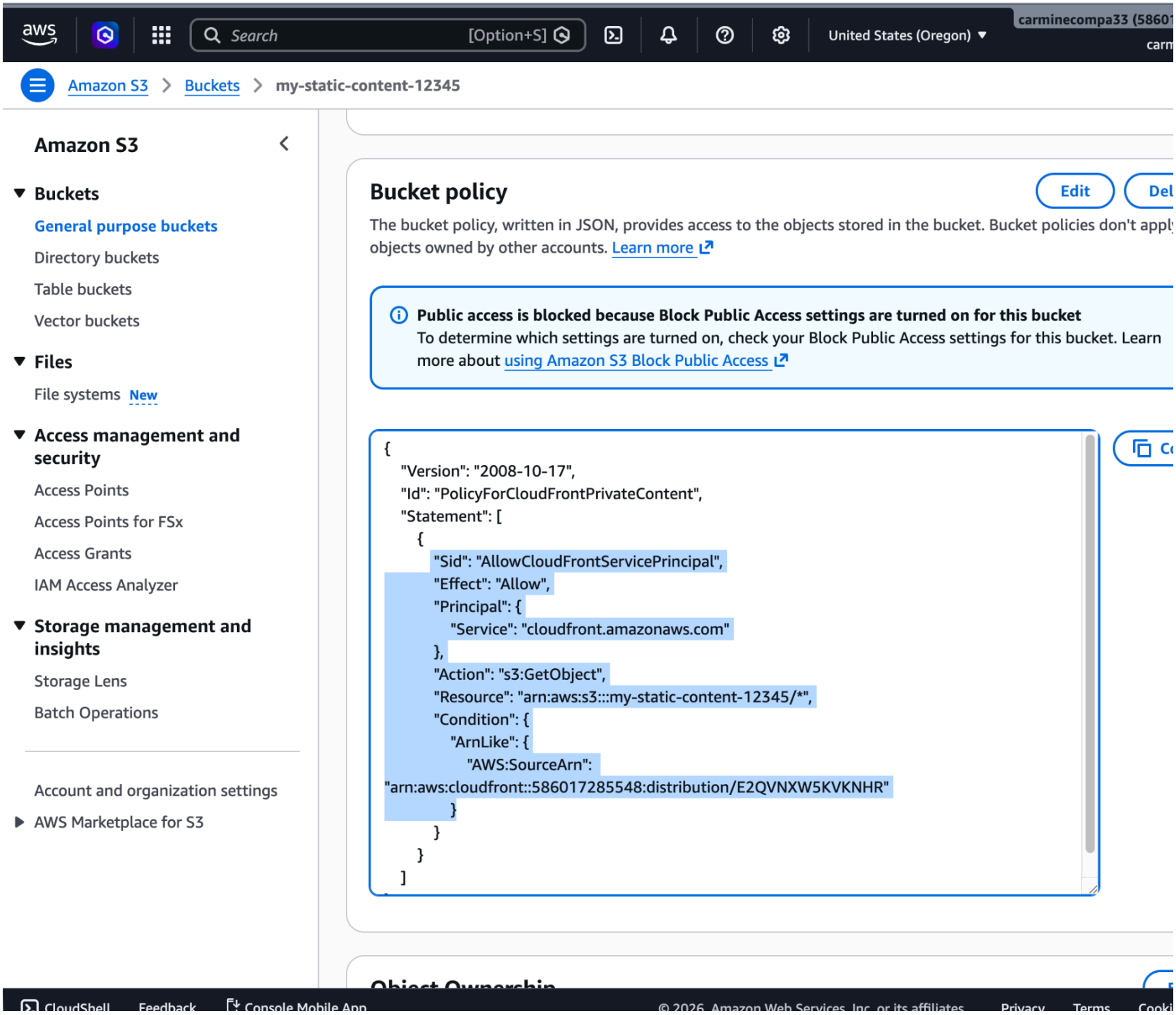
Mount & Test: Run the provided network mounting command on your local machine to turn the cloud gateway into a local folder. Save a test file there and watch it immediately appear in your S3 bucket.

Part 1: S3 & CloudFront (Global Content Delivery)

Created S3 bucket with an index.html file



Created a cloudfront distribution



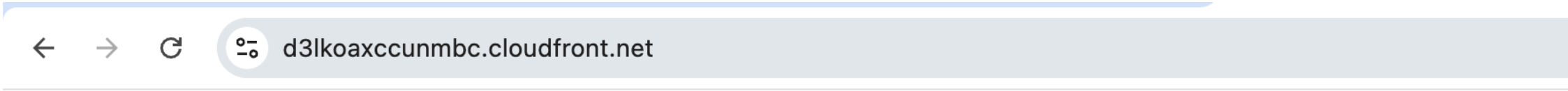
My S3 bucket cannot be accessed by the internet by defauly



This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
▼<Error>
   <Code>AccessDenied</Code>
   <Message>Access Denied</Message>
   <RequestId>HKBWDQHA20TEQ1JH</RequestId>
   <HostId>rz+VkbxC4eOjZwli178SbvwgBPUX518Knn9rQP6wUz393eeGD8YGqm10FxmZIw0Xw/SSkagsPZllNoiQfWLNRMqm0+VE2bodHIG4+v44Z6Q=</HostId>
 </Error>
```

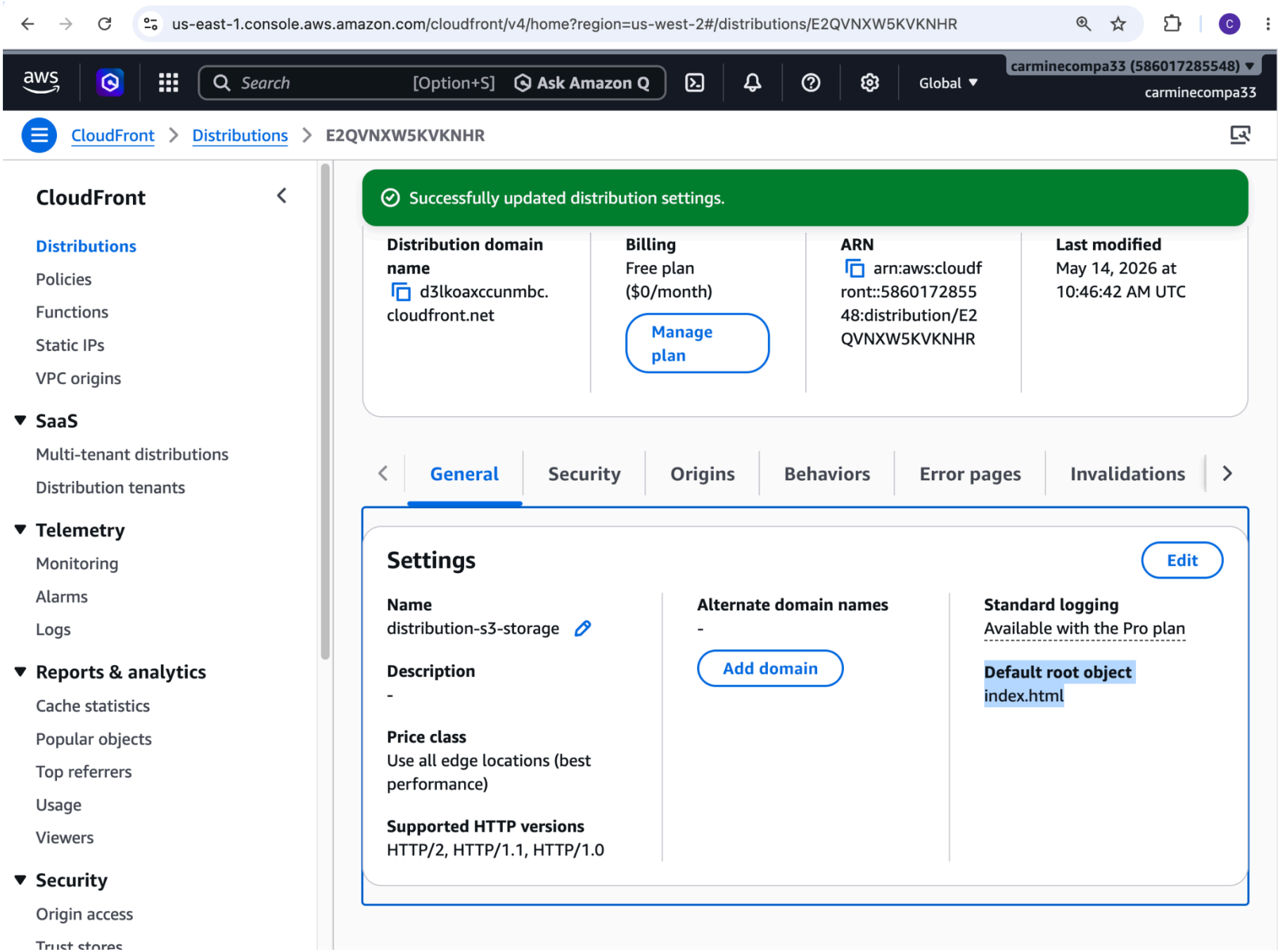

But also my cloudfront distribution:



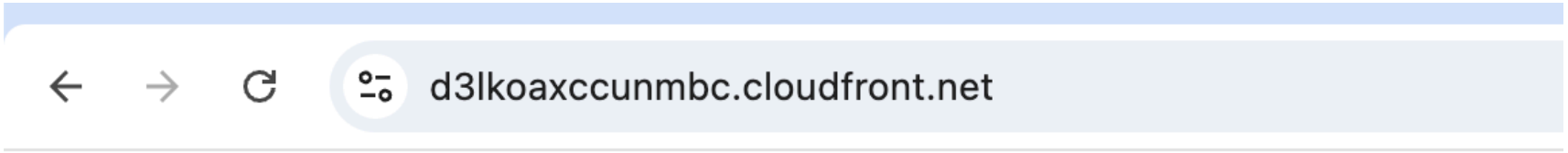
This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
▼<Error>
   <Code>AccessDenied</Code>
   <Message>Access Denied</Message>
 </Error>
```

I add index.html as default root object

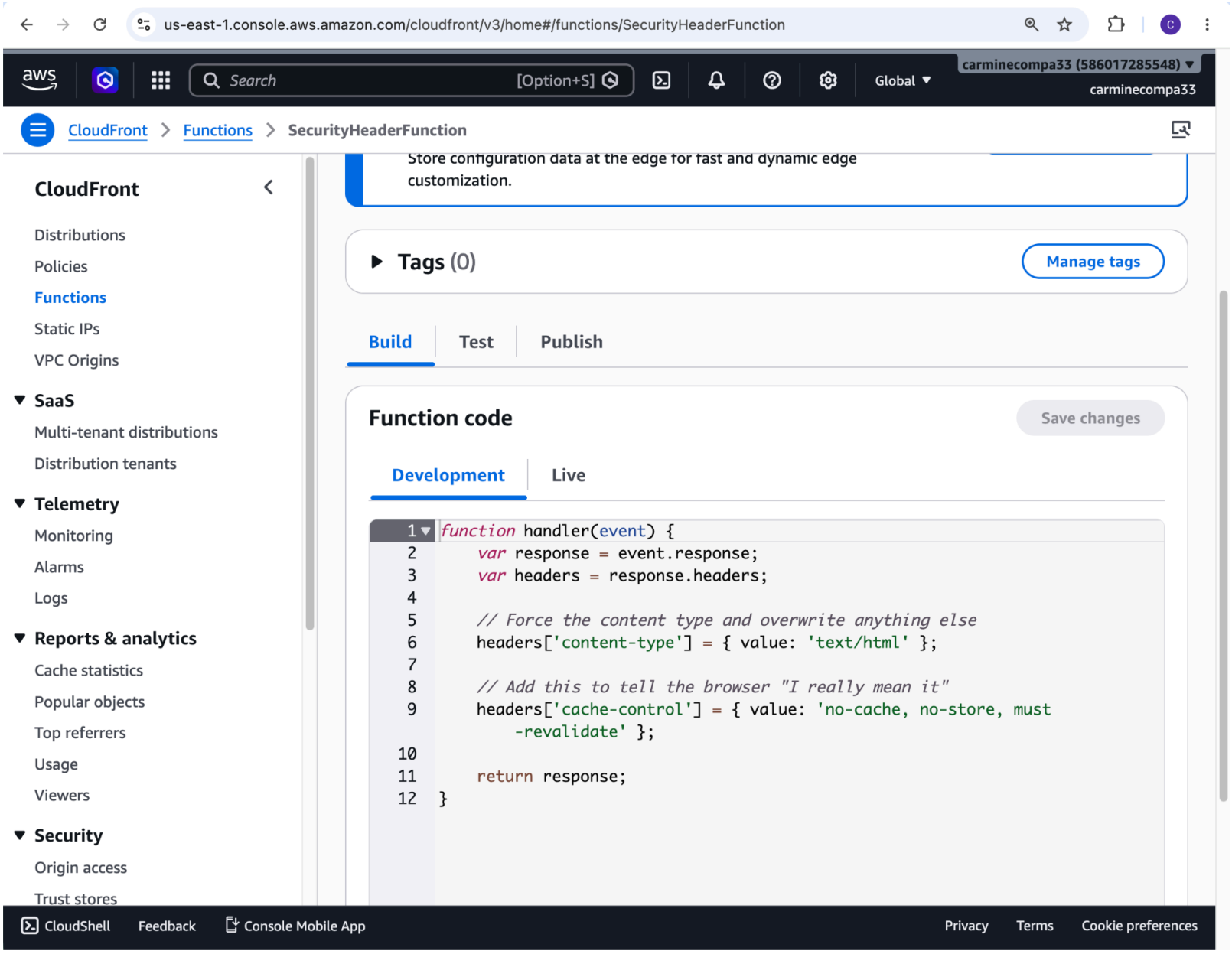


However, cloudfront distribution is still not accessible



<html> <body> <h1>Welcome to my Private S3 Site</h1> </body> </html>

I force to have text/html as content-type header with a cloudfront function



I associate the cloudfront function to the cloudfront distribution behavior viewer response

View policy

Origin request policy - *optional*

Choose an existing origin request policy or create a new one.

Select origin policy

Response headers policy - *optional*

Choose an existing response headers policy or create a new one.

Select response headers

▶ **Additional settings**

Function associations - *optional* Info

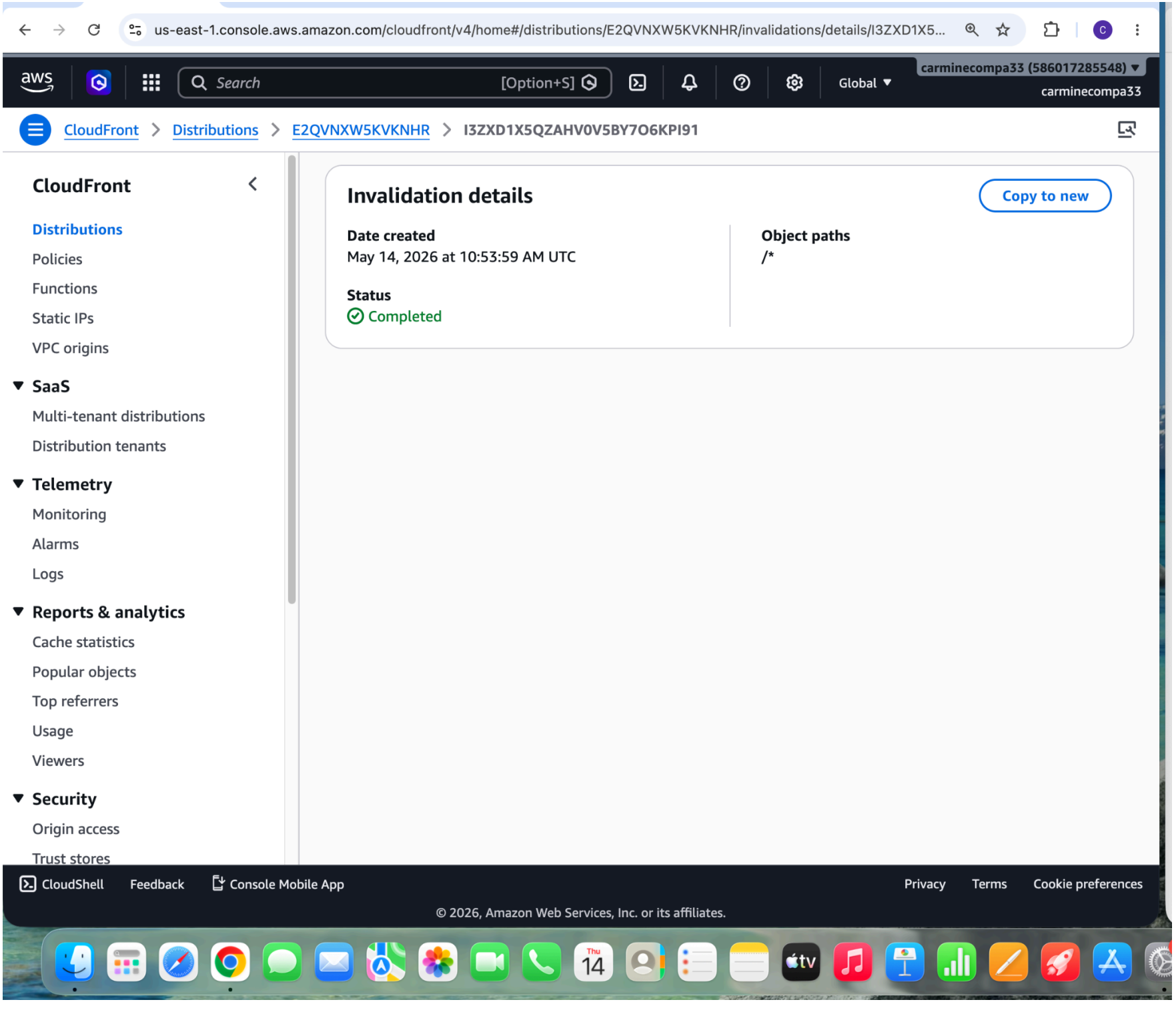
Choose an edge function to associate with this cache behavior, and the CloudFront event that invokes the function.

	Function type	Function ARN / Name	Include body
Viewer request	No association		
Viewer response	CloudFront Functions	SecurityHeaderFunction	
Origin request	No association		
Origin response	No association		

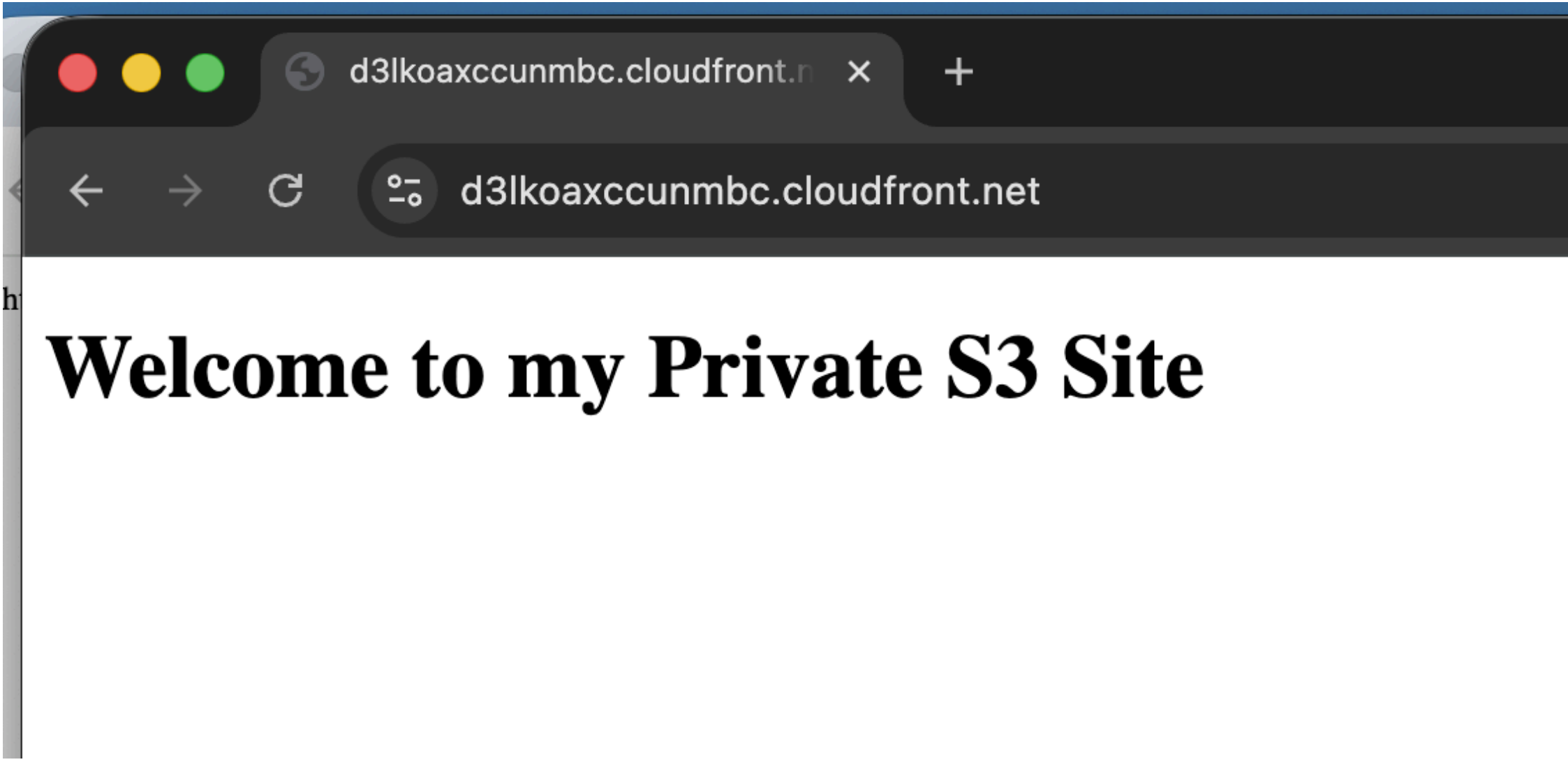
Cancel Save changes

© 2026, Amazon Web Services, Inc. or its affiliates.

I invalidate/clear cache in order to take new version of index.html

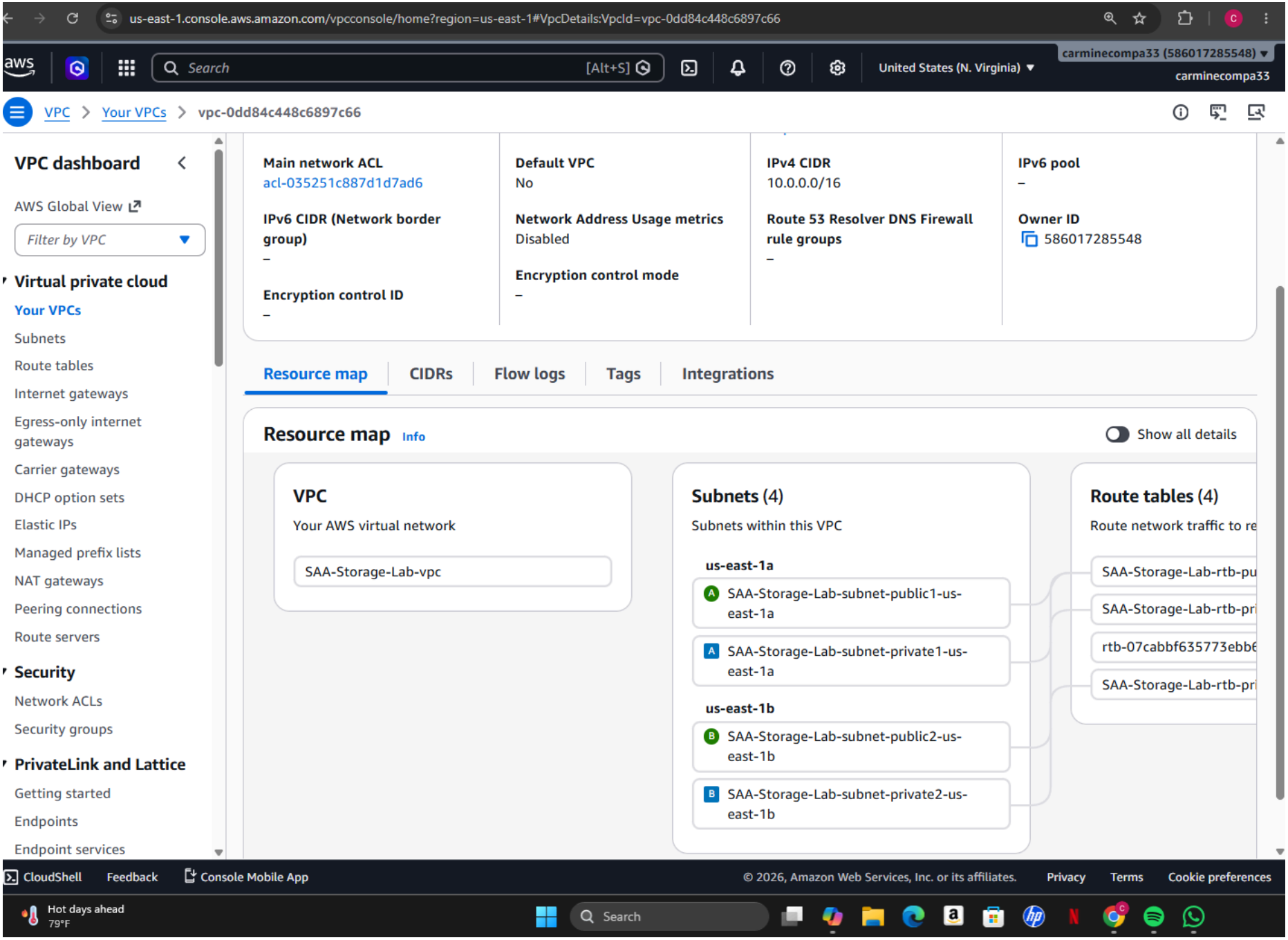


Now my index.html is accessible via cloudfront

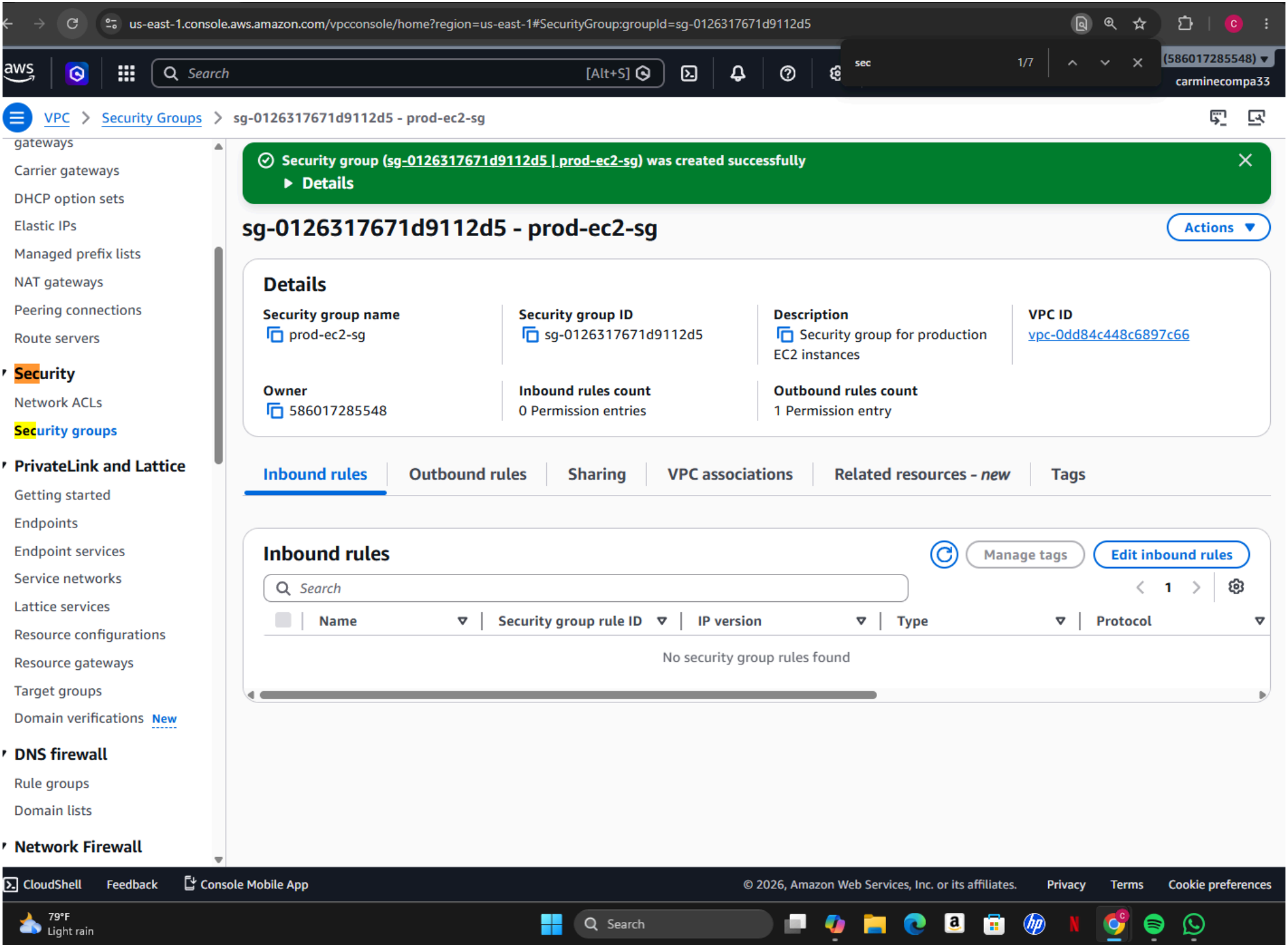


Part 2: Amazon EFS (Shared Regional Storage)

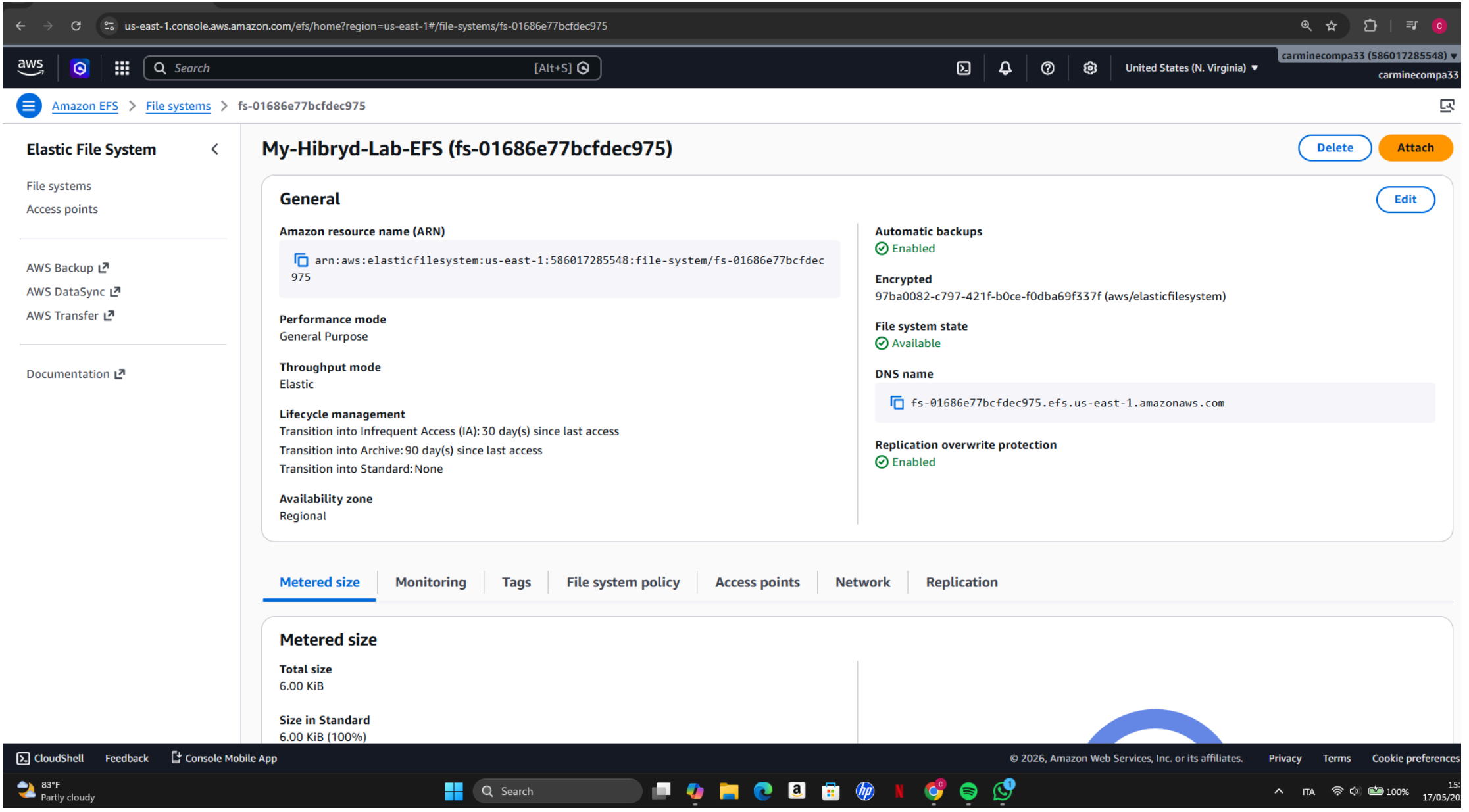
I create a VPC with two Availability Zones (east-1a and east-1b)



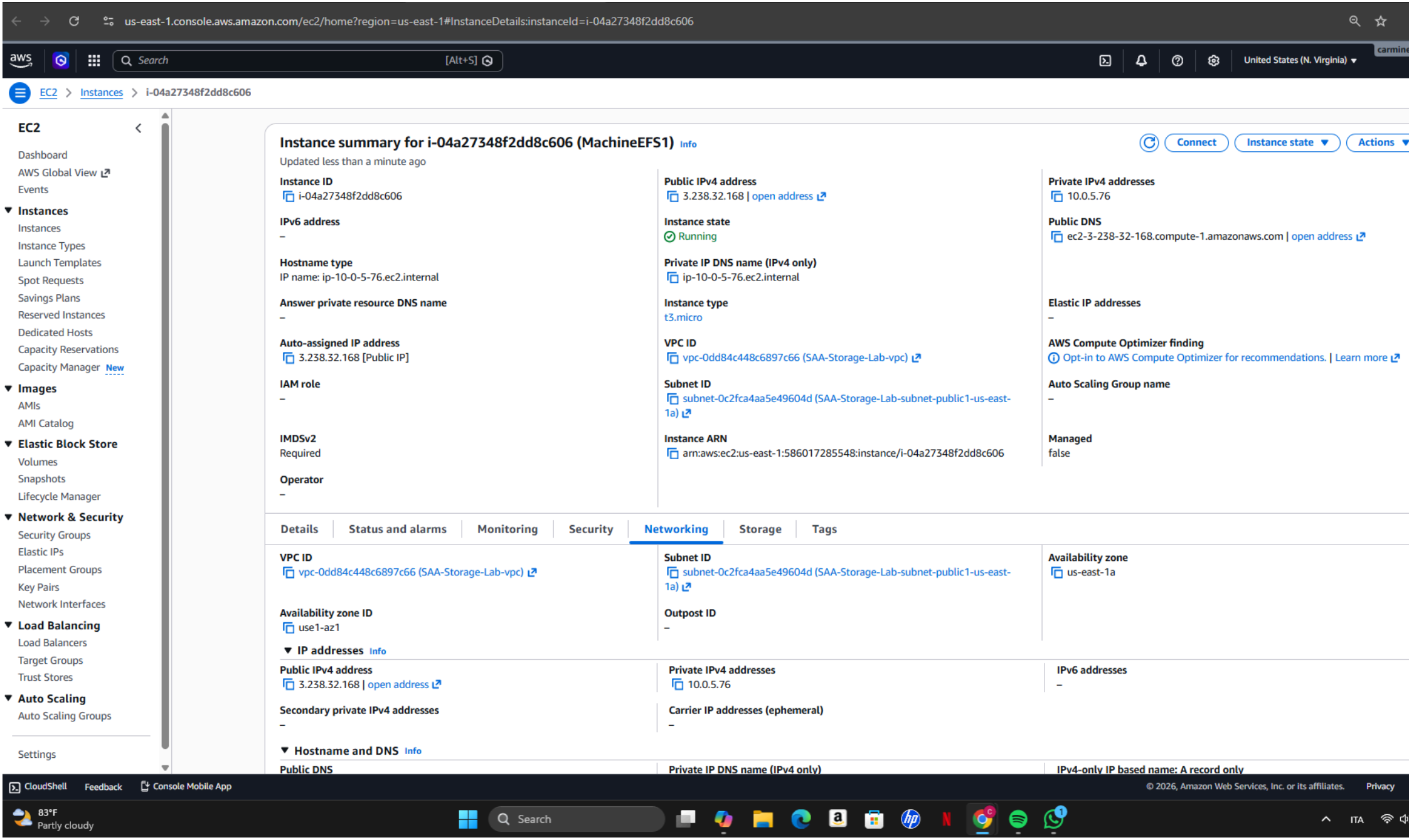
I create Security Group that will be attached to the EC2



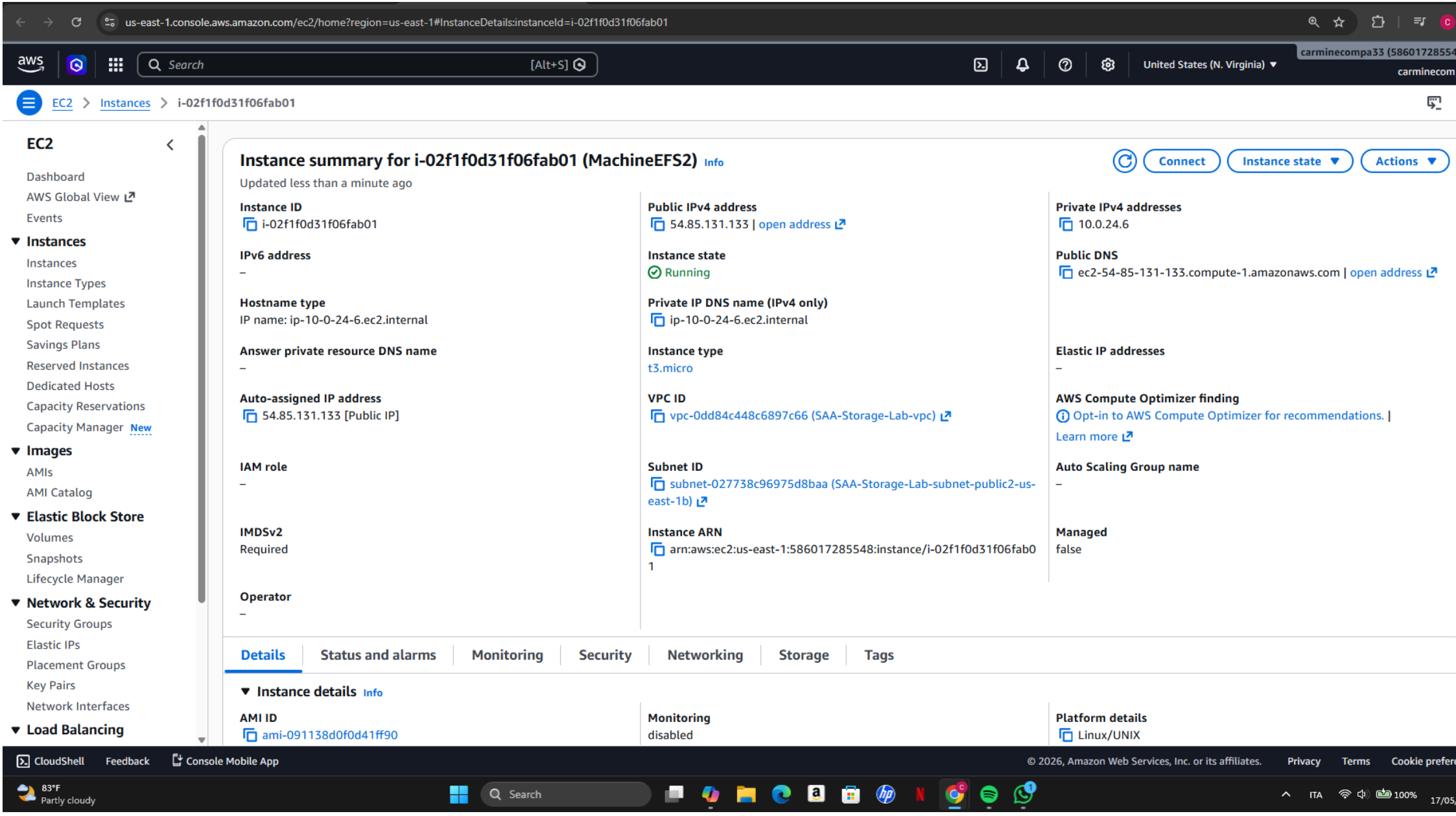
Created Elastic File System - Availability Zone: standard



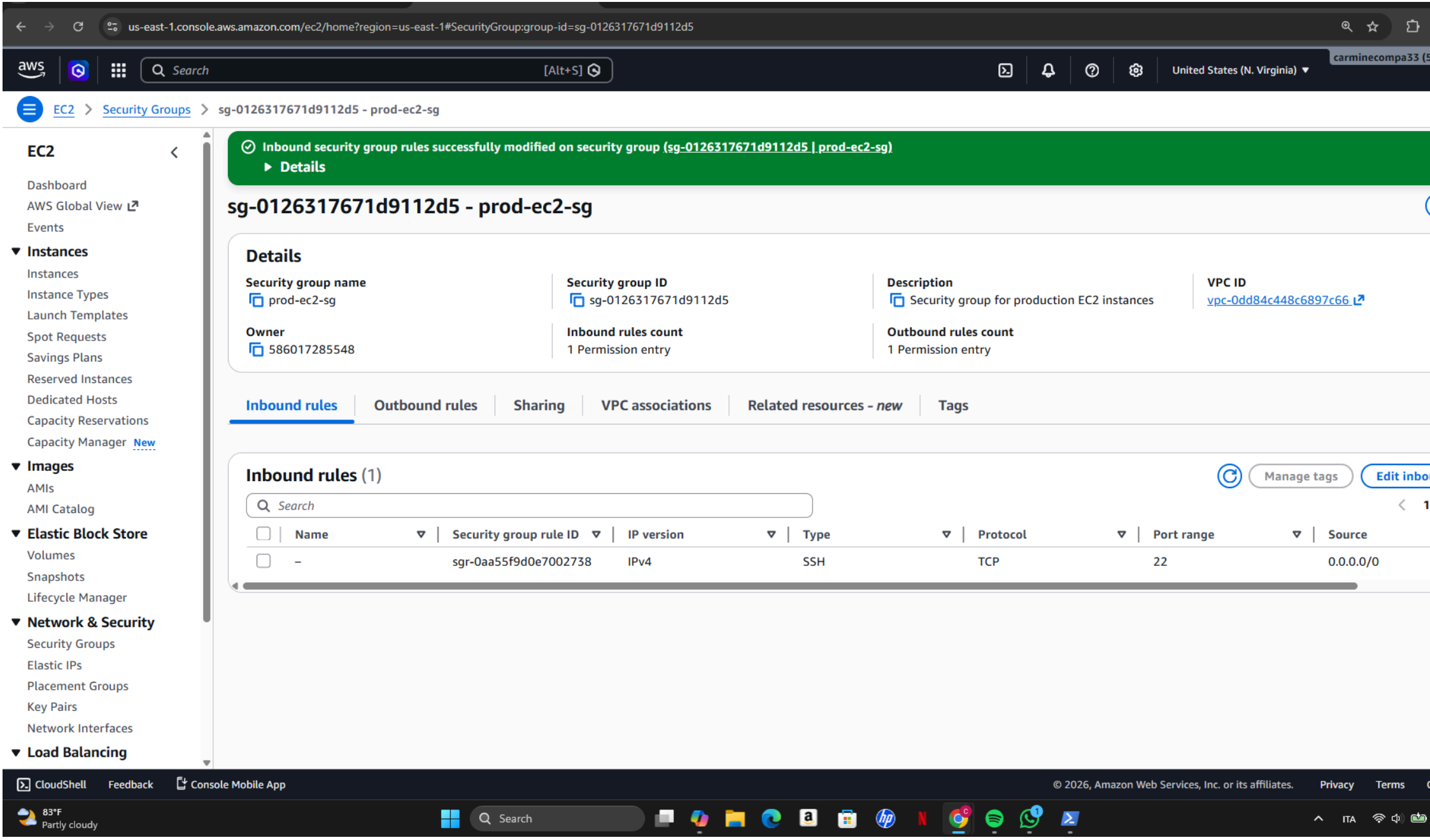
Created first EC2 instance with Public subnet us-east-1a Security Group: prod-ec2-sg



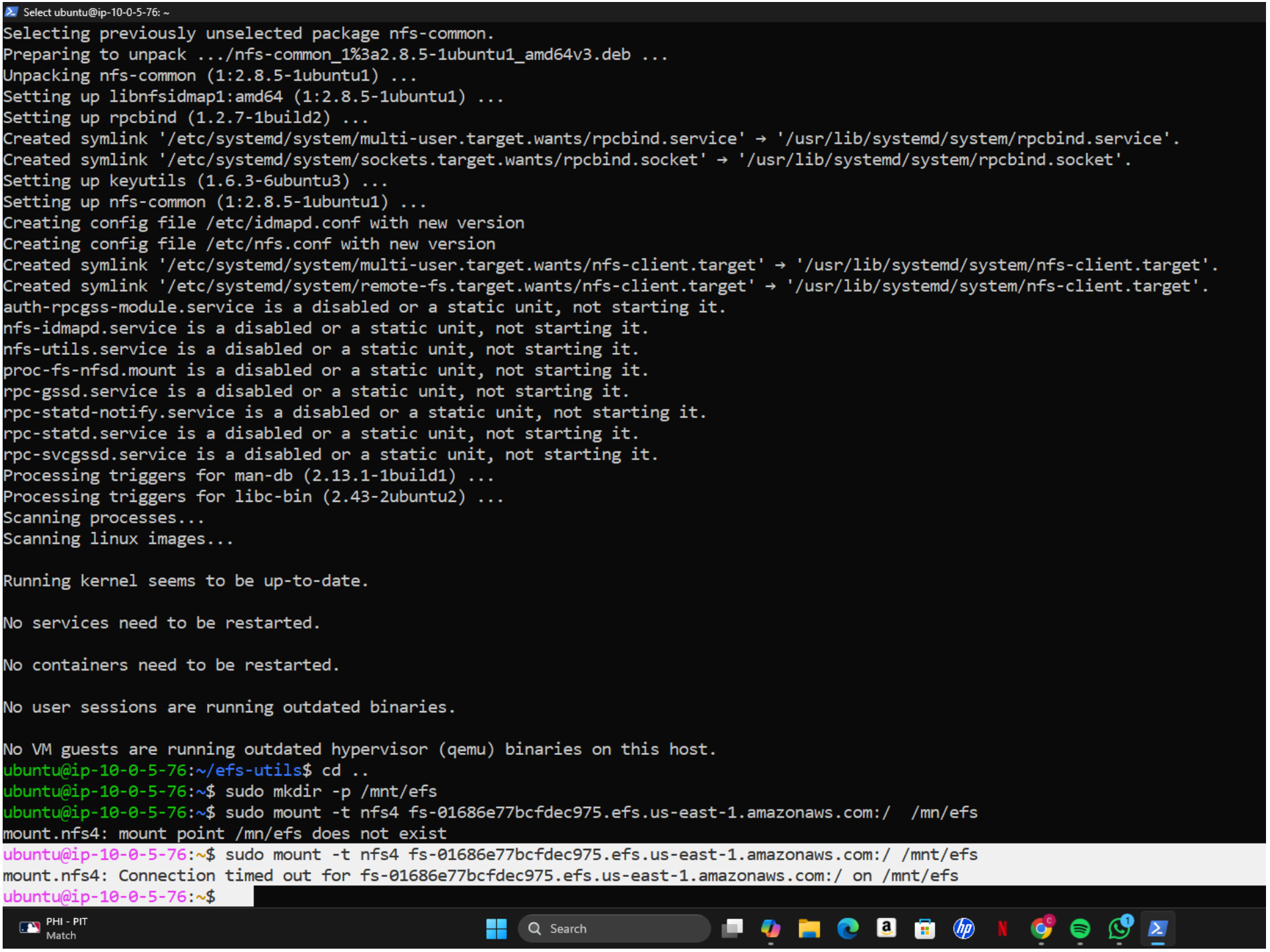
Created second EC2 with: Public subnet in us-east-1b Security Group prod-ec2-sg



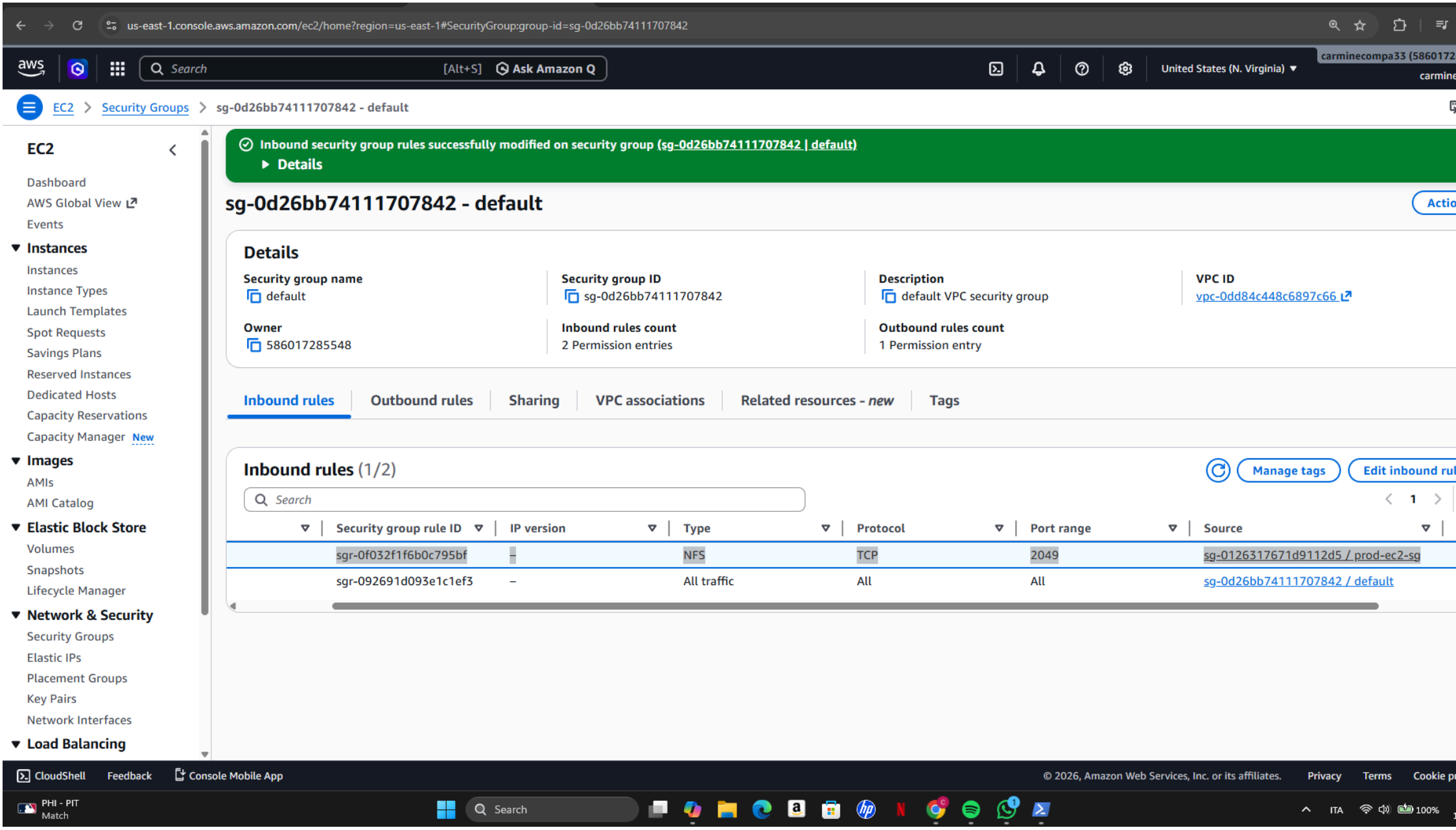
Open the Security Group to inbound SSH connections



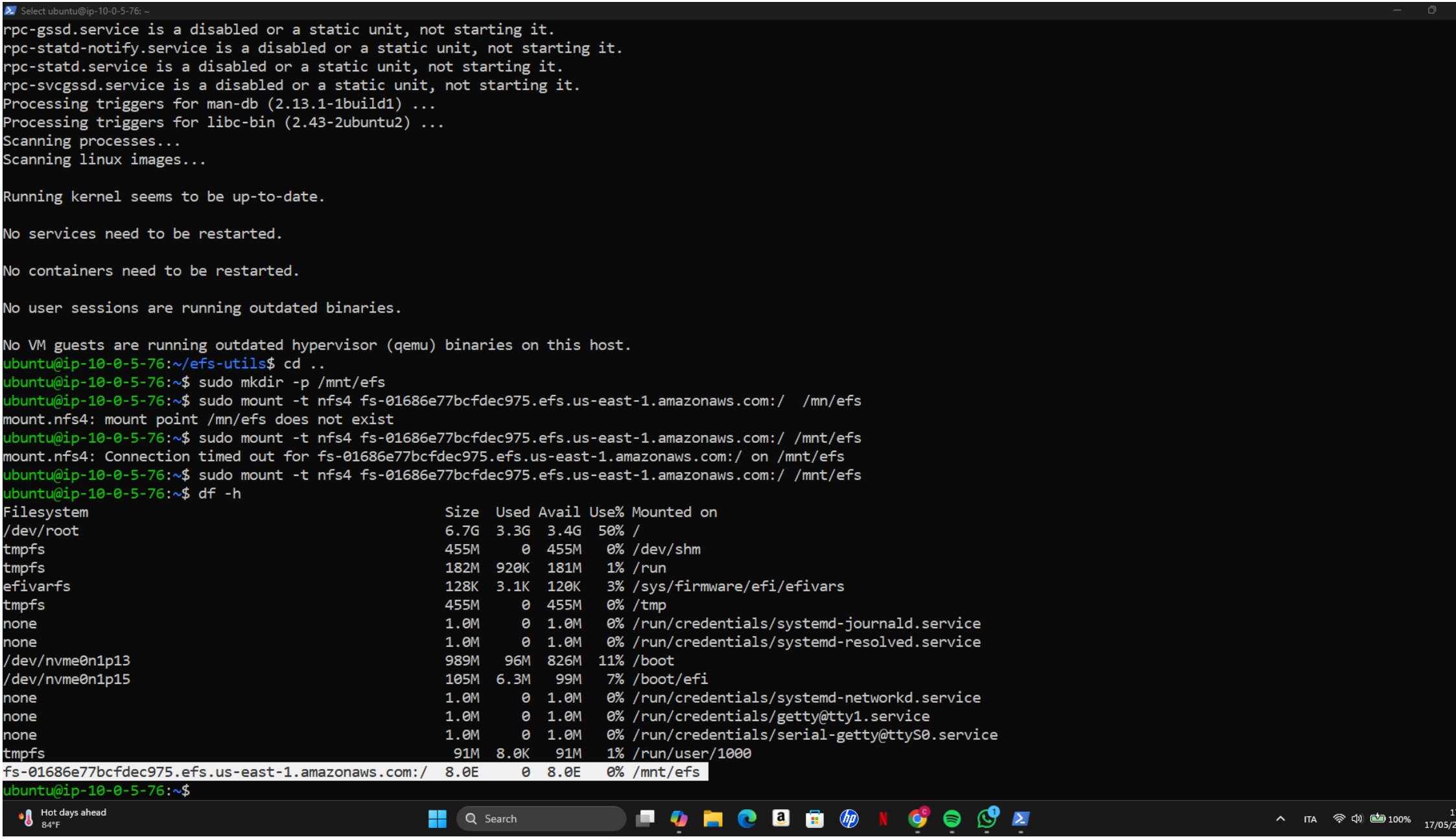
Sudo mount is timing out



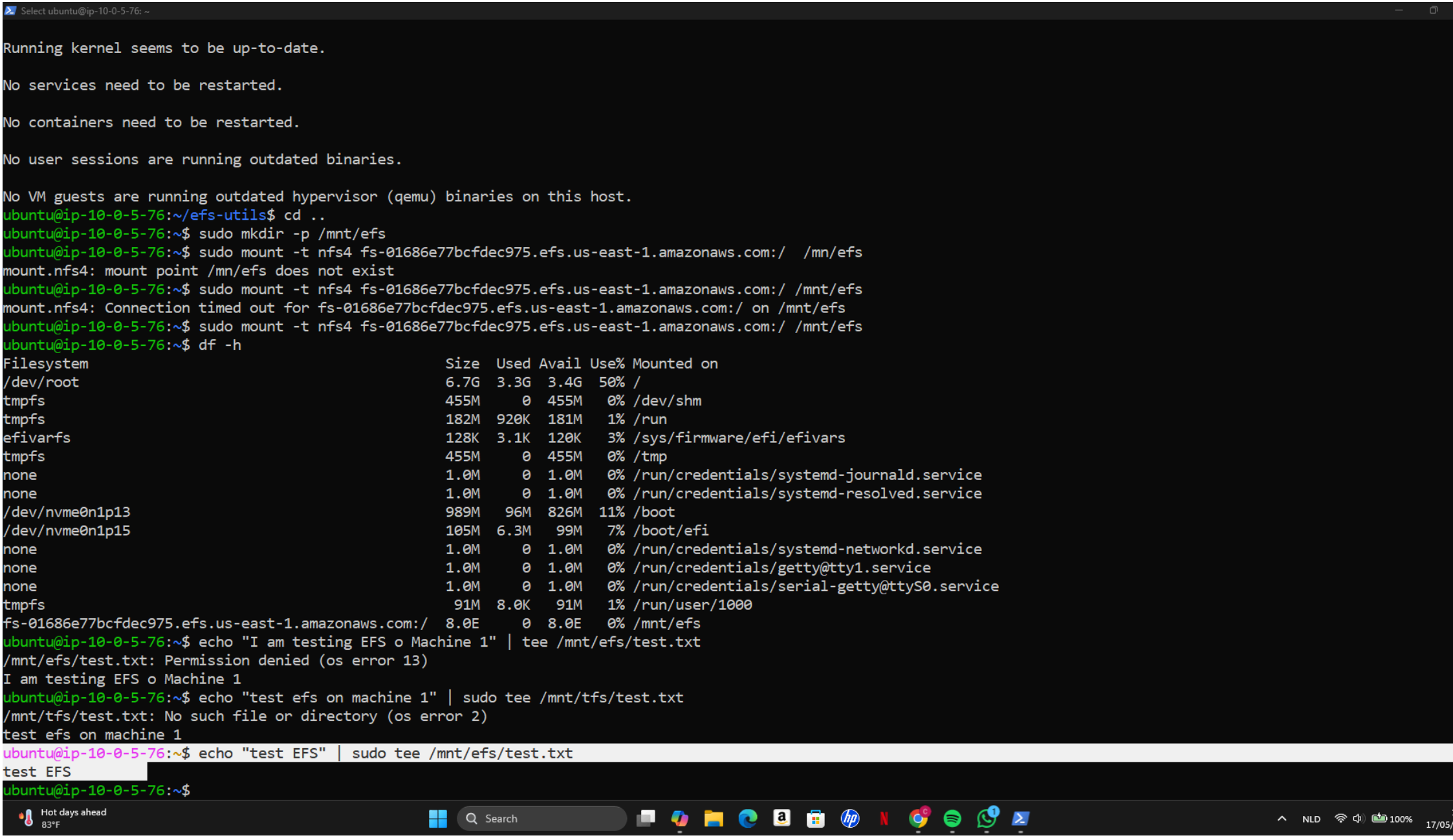
I open port 2049 on the Elastic file system Security Group only if inbound connection comes from EC2 Security Group (Security Group Nesting or Chaining)



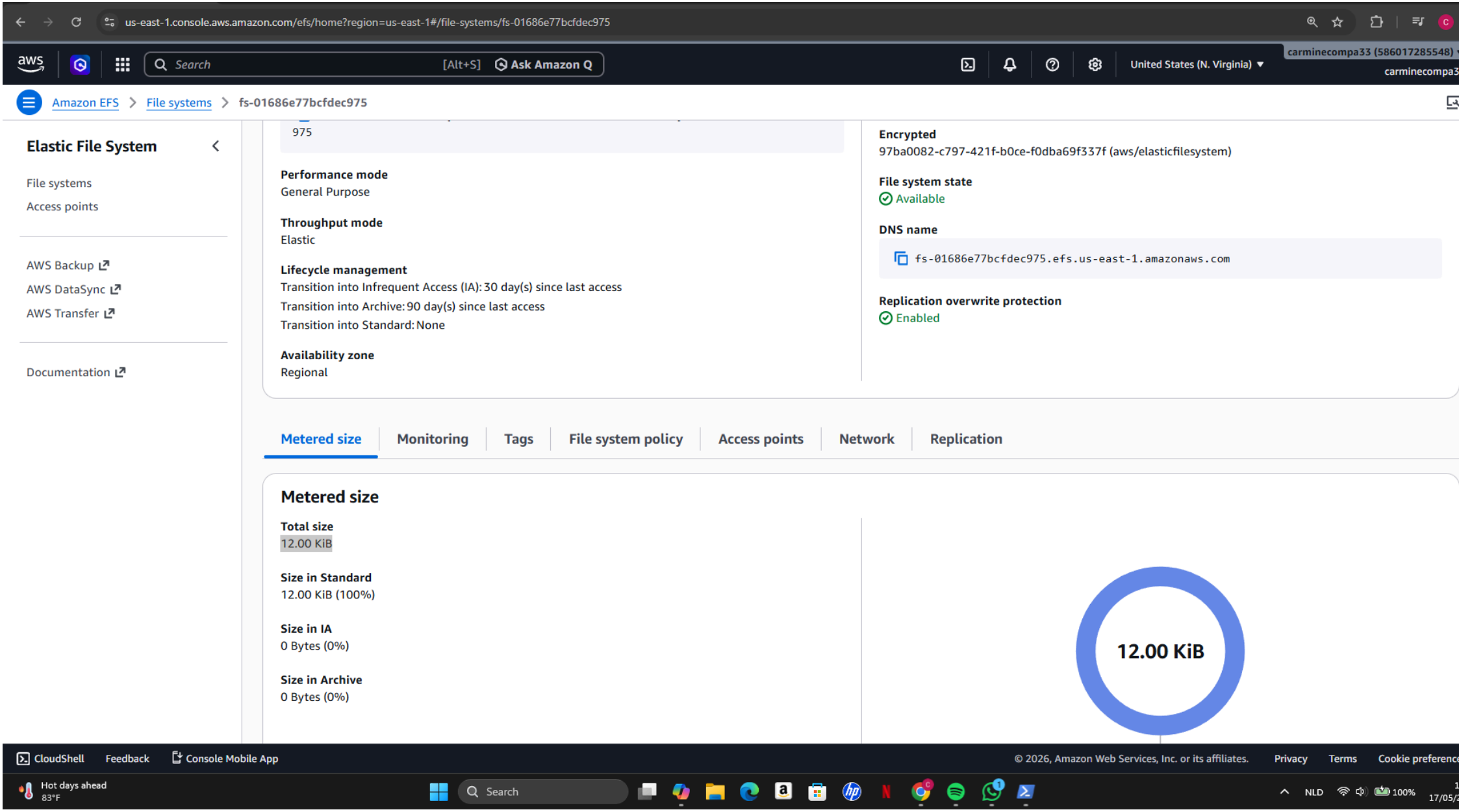
Now EFS mount works correctly: Size is 8GB but but scalable to virtually endless size



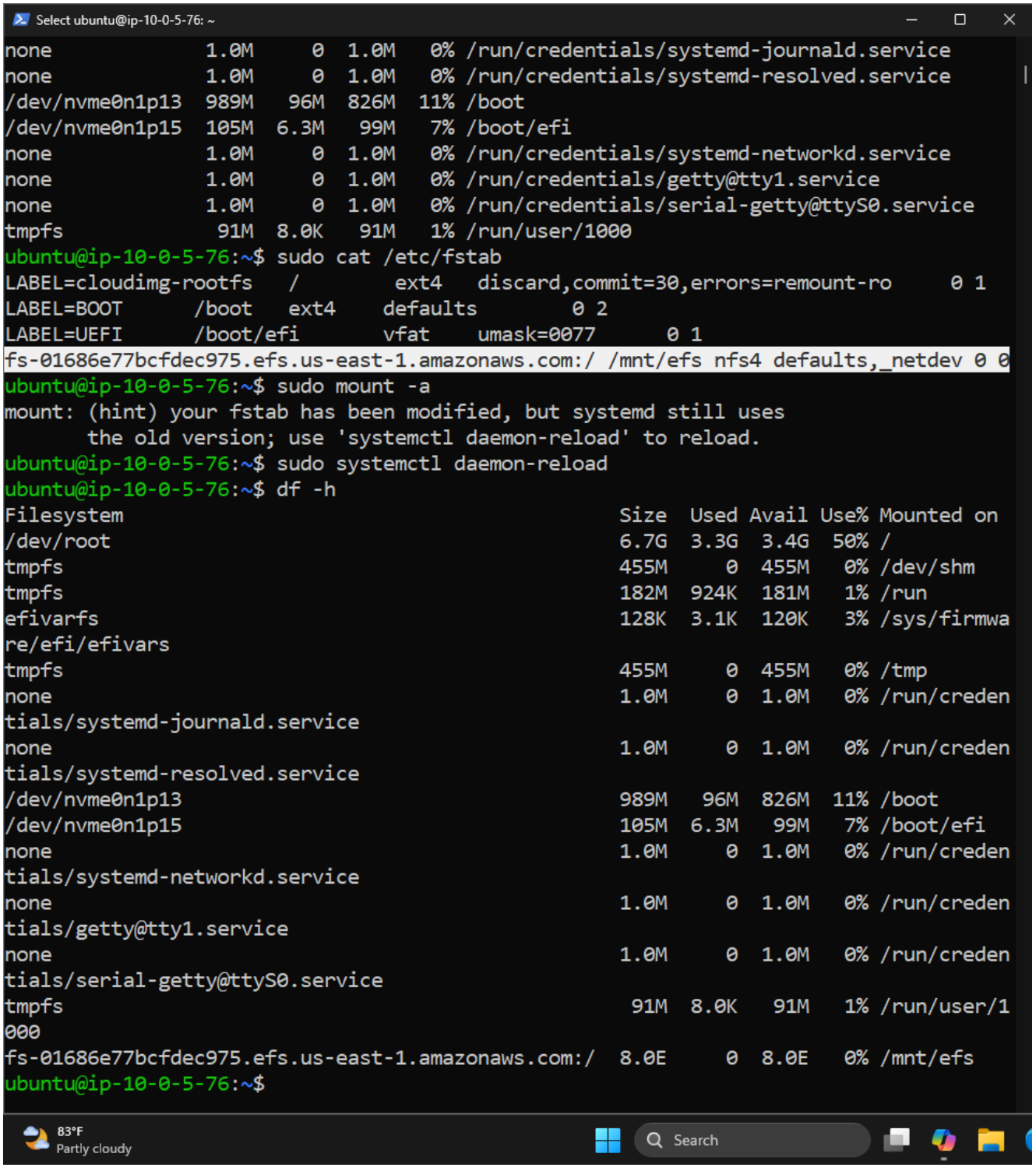
I am able to copy data in the EFS



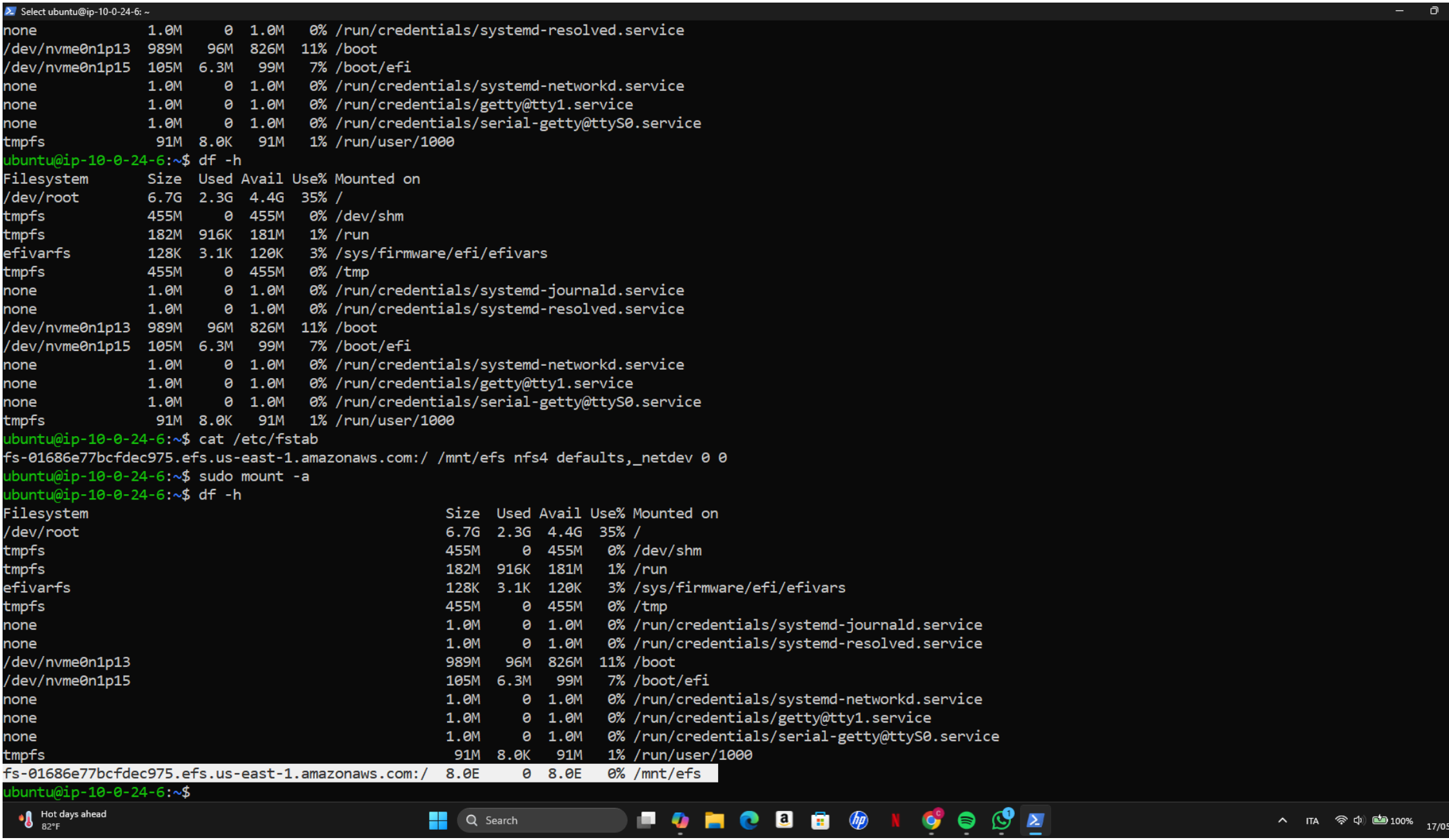
Now I can see the test.txt in my EFS in AWS with the 12kb



Added /mnt/efs to the boot of the EC2 by adding the elastic file system to the /etc/fstab
Sudo systemctl daemon-reload loads the change in the /etc/fstab



Added the same configuration of the EFS on the folder /mnt/efs on the second EC2 as well



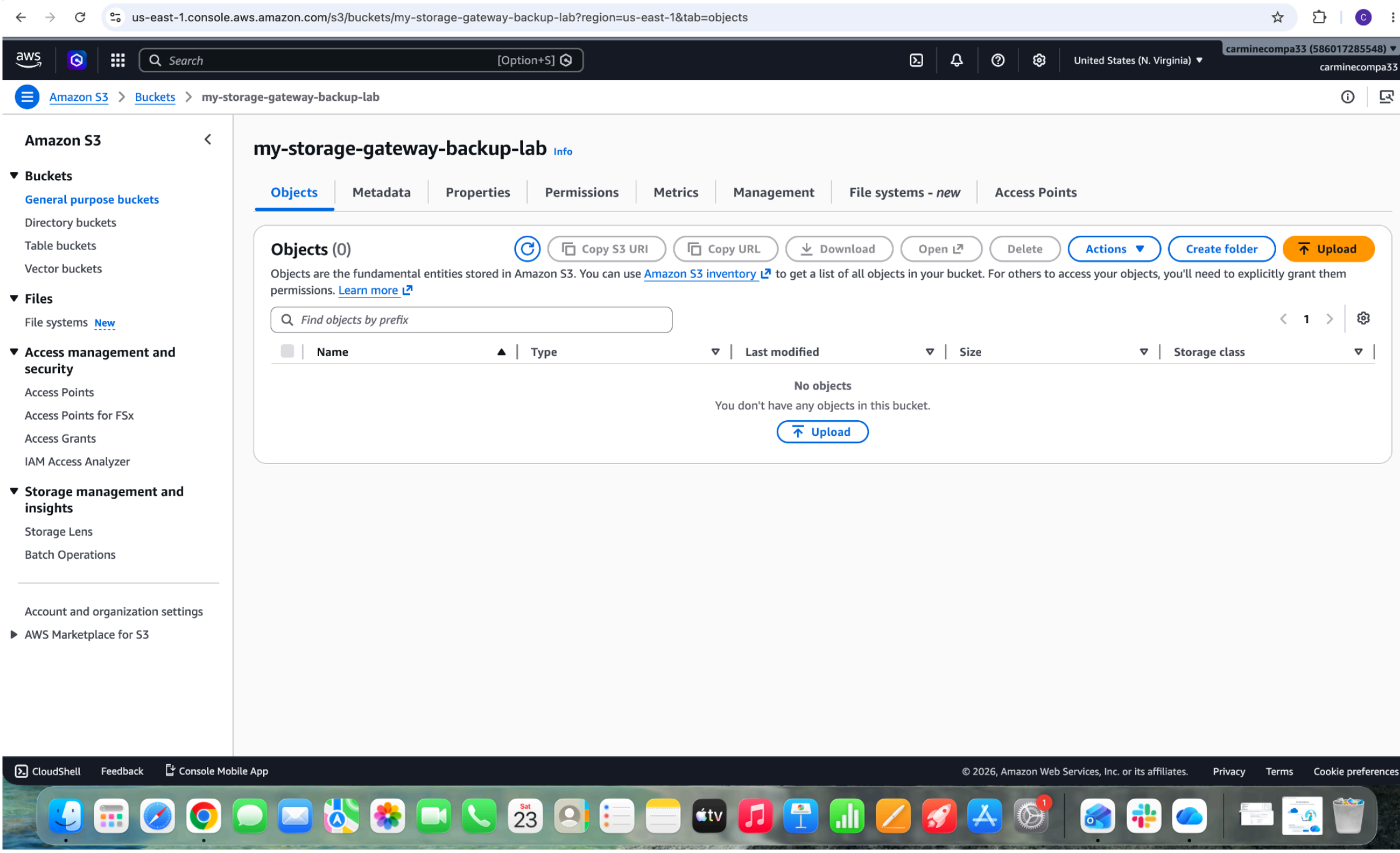
Ultimate File System check; I have visibility on the second EC2 on test.txt that was created in the EFS from the 1st EC2

```
Select ubuntu@ip-10-0-24-6: ~
none             1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
none             1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyS0.service
tmpfs             91M  8.0K   91M   1% /run/user/1000
ubuntu@ip-10-0-24-6:~$ df -h
Filesystem       Size  Used Avail Use% Mounted on
/dev/root        6.7G  2.3G  4.4G  35% /
tmpfs            455M     0  455M   0% /dev/shm
tmpfs            182M  916K  181M   1% /run
efivarfs         128K  3.1K  120K   3% /sys/firmware/efi/efivars
tmpfs            455M     0  455M   0% /tmp
none             1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none             1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/nvme0n1p13  989M   96M  826M  11% /boot
/dev/nvme0n1p15  105M  6.3M   99M   7% /boot/efi
none             1.0M     0  1.0M   0% /run/credentials/systemd-networkd.service
none             1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
none             1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyS0.service
tmpfs             91M  8.0K   91M   1% /run/user/1000
ubuntu@ip-10-0-24-6:~$ cat /etc/fstab
fs-01686e77bcfdec975.efs.us-east-1.amazonaws.com:/ /mnt/efs nfs4 defaults,_netdev 0 0
ubuntu@ip-10-0-24-6:~$ sudo mount -a
ubuntu@ip-10-0-24-6:~$ df -h
Filesystem                                      Size  Used Avail Use% Mounted on
/dev/root                                       6.7G  2.3G  4.4G  35% /
tmpfs                                           455M     0  455M   0% /dev/shm
tmpfs                                           182M  916K  181M   1% /run
efivarfs                                        128K  3.1K  120K   3% /sys/firmware/efi/efivars
tmpfs                                           455M     0  455M   0% /tmp
none                                            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none                                            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/nvme0n1p13                                 989M   96M  826M  11% /boot
/dev/nvme0n1p15                                 105M  6.3M   99M   7% /boot/efi
none                                            1.0M     0  1.0M   0% /run/credentials/systemd-networkd.service
none                                            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
none                                            1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyS0.service
tmpfs                                            91M  8.0K   91M   1% /run/user/1000
fs-01686e77bcfdec975.efs.us-east-1.amazonaws.com:/  8.0E     0  8.0E   0% /mnt/efs
ubuntu@ip-10-0-24-6:~$ cat /mnt/efs/test.txt
test EFS
ubuntu@ip-10-0-24-6:~$ sudo ls /mnt/efs
test.txt
ubuntu@ip-10-0-24-6:~$
```

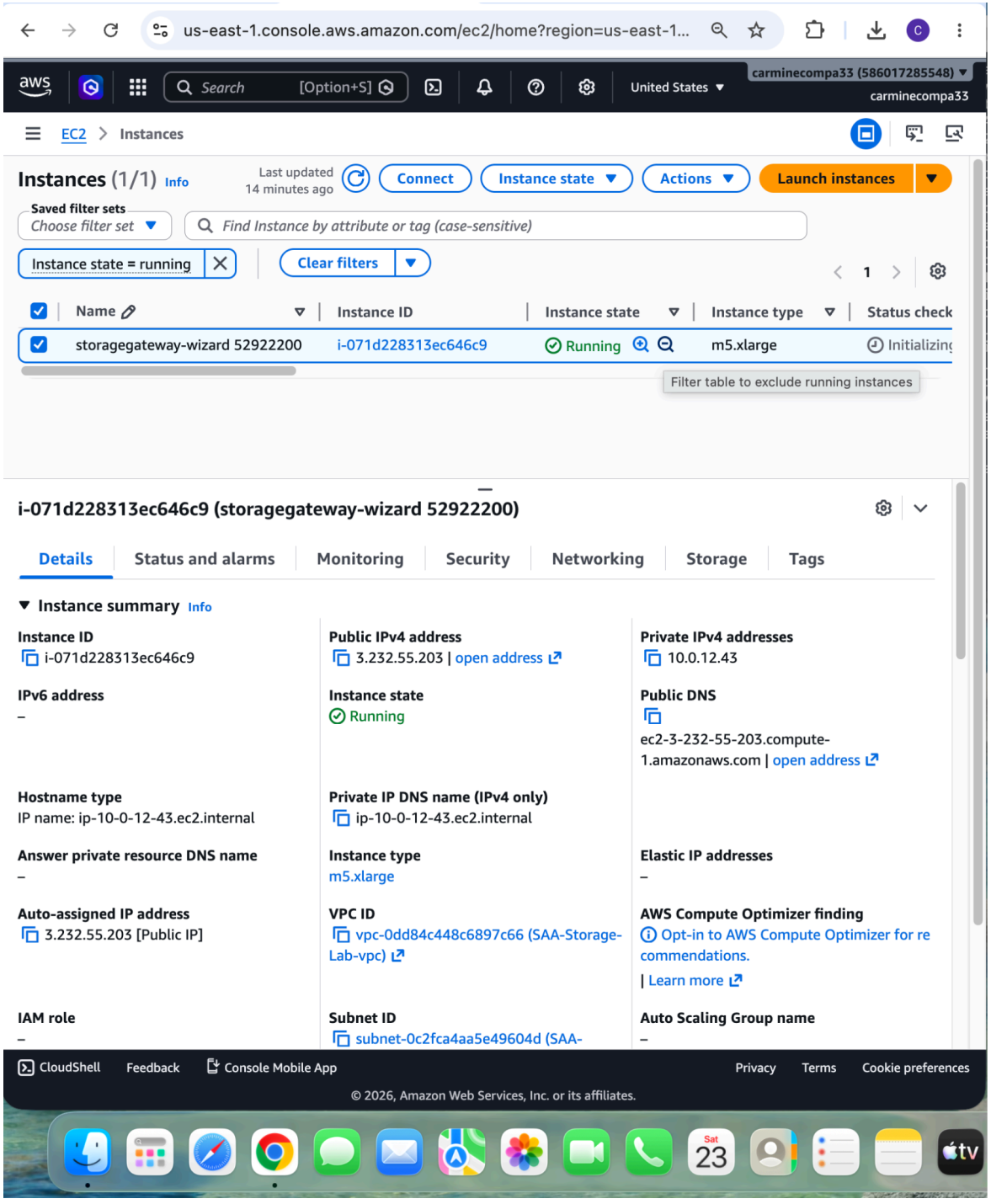
Hot days ahead
82°F
Search
ITA
100%
17/05

Part 3: Storage Gateway (Architectural Knowledge)

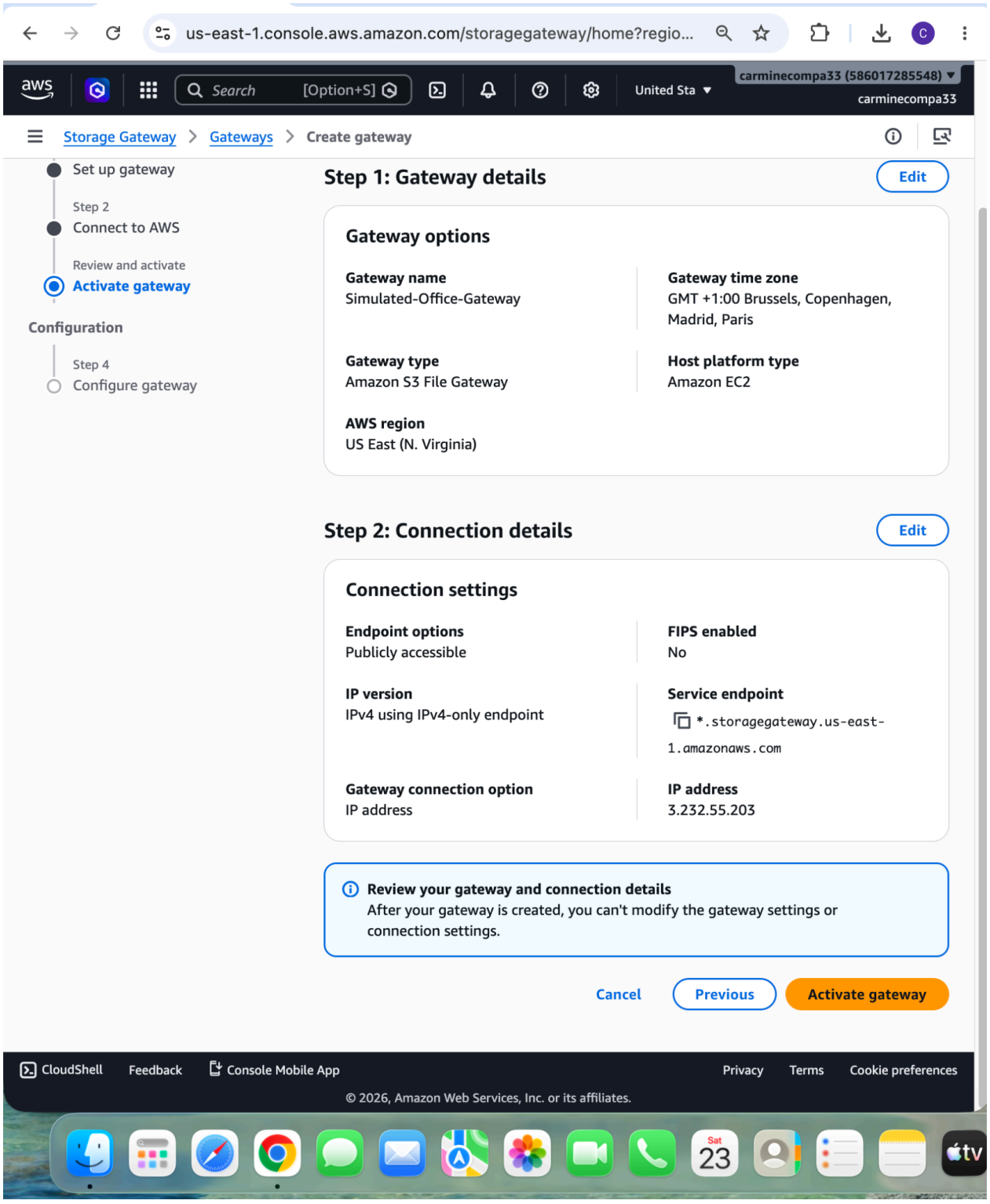
I create an S3 Bucket



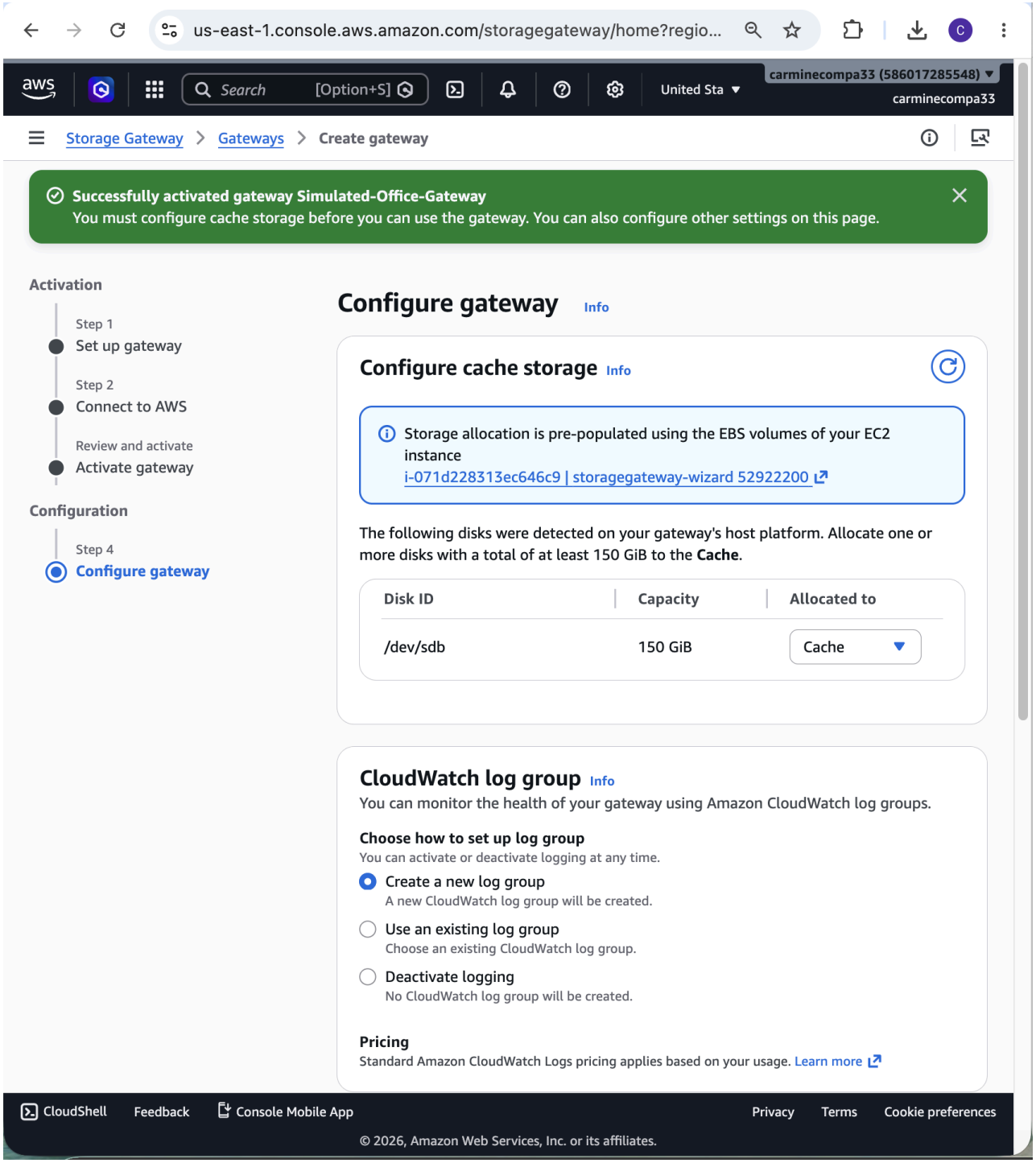
Create large EC2 instance that will simulate our “office storage server”



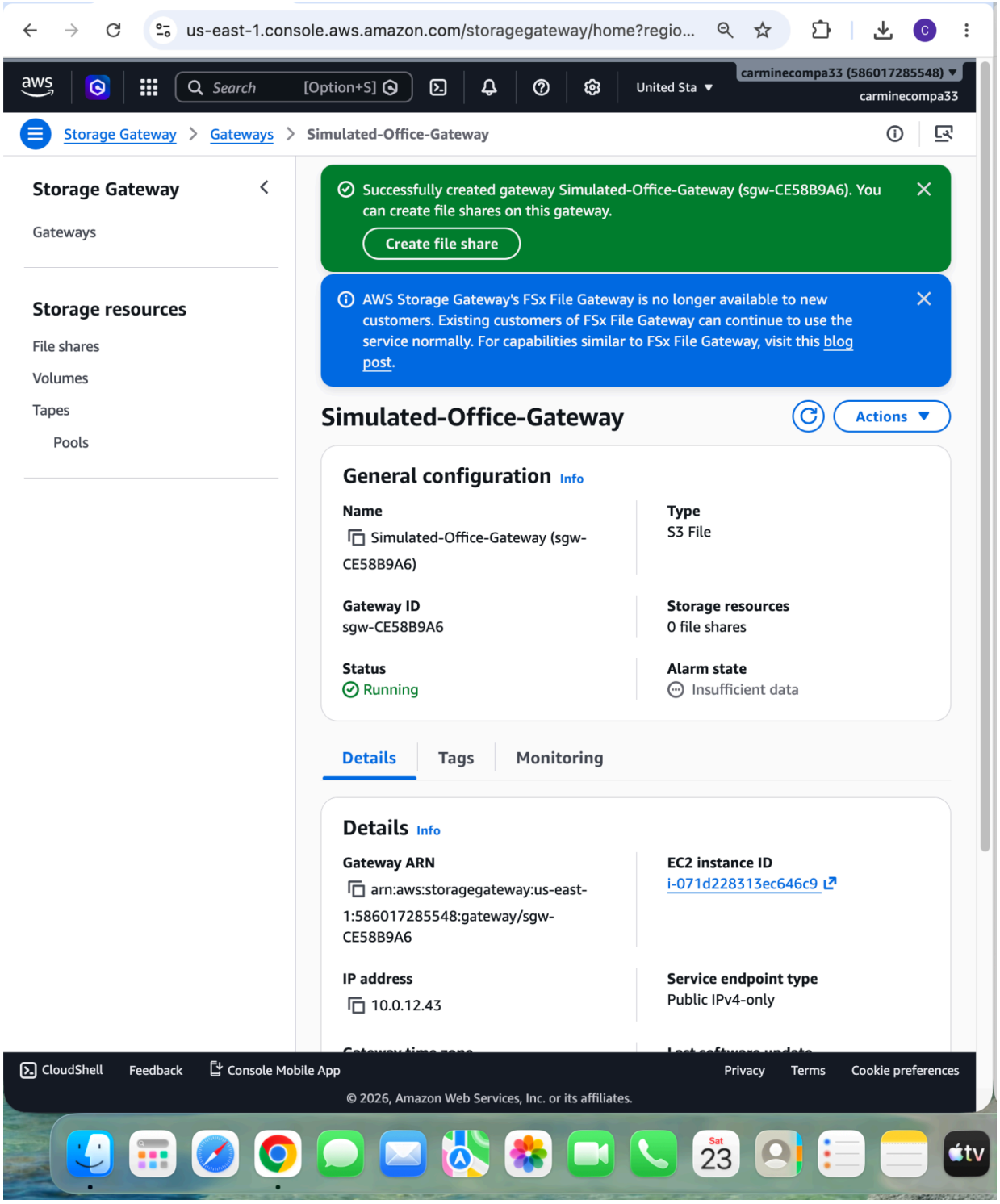
AWS Storage Gateway can connect to the EC2 instance


us-east-1.console.aws.amazon.com/storagegateway/home?regio...
aws
Search
[Option+S]
United Sta
carminecompa33 (586017285548)
carminecompa33
Storage Gateway
Gateways
Create gateway
Set up gateway
Step 2
Connect to AWS
Review and activate
Activate gateway
Configuration
Step 4
Configure gateway
Step 1: Gateway details
Edit
Gateway options
Gateway name
Simulated-Office-Gateway
Gateway time zone
GMT +1:00 Brussels, Copenhagen, Madrid, Paris
Gateway type
Amazon S3 File Gateway
Host platform type
Amazon EC2
AWS region
US East (N. Virginia)
Step 2: Connection details
Edit
Connection settings
Endpoint options
Publicly accessible
FIPS enabled
No
IP version
IPv4 using IPv4-only endpoint
Service endpoint
*.storagegateway.us-east-1.amazonaws.com
Gateway connection option
IP address
IP address
3.232.55.203
Review your gateway and connection details
After your gateway is created, you can't modify the gateway settings or connection settings.
Cancel
Previous
Activate gateway
CloudShell
Feedback
Console Mobile App
Privacy
Terms
Cookie preferences
© 2026, Amazon Web Services, Inc. or its affiliates.

Successfully activated the storage gateway

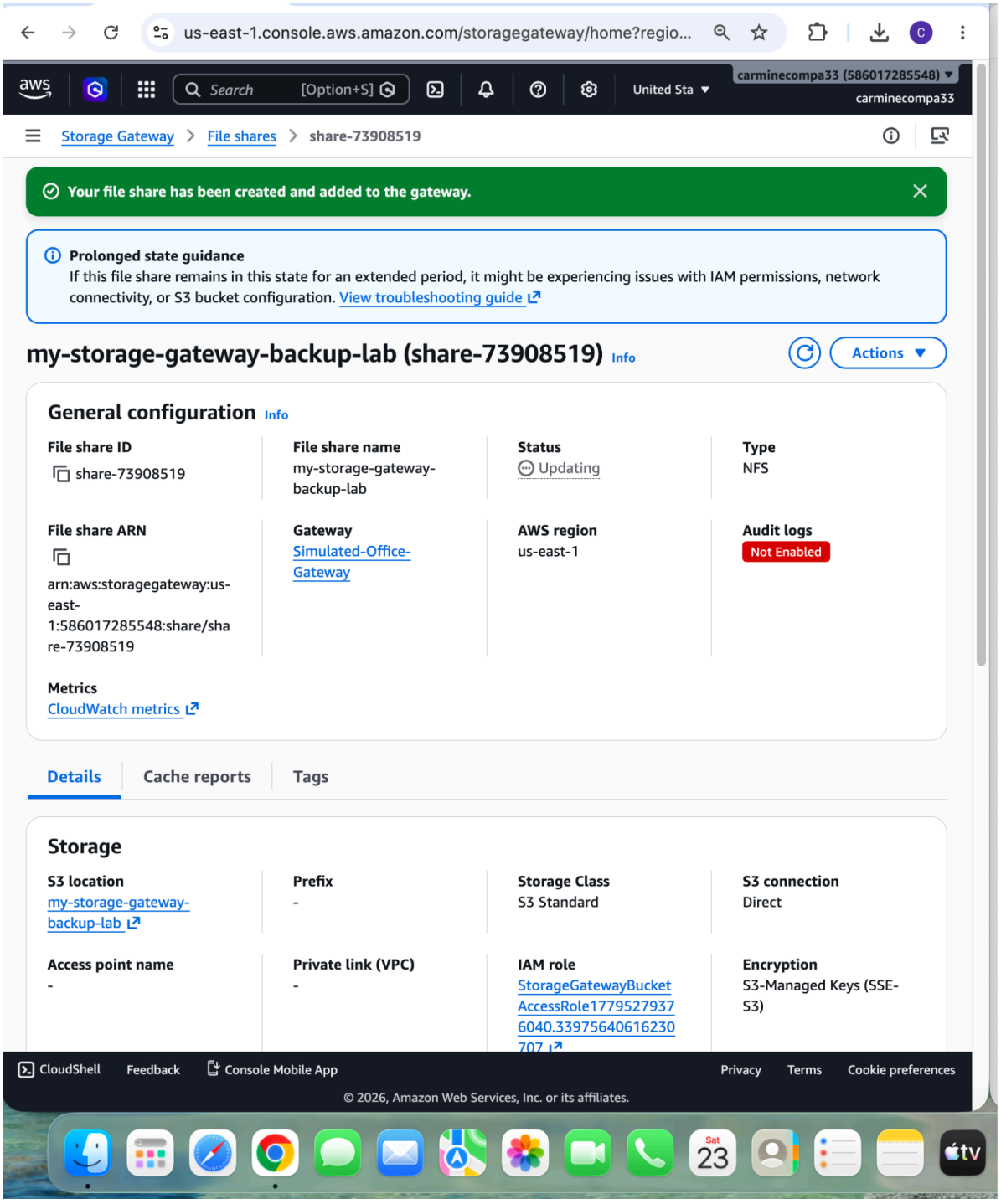


Completed the creation of the storage gateway which now running

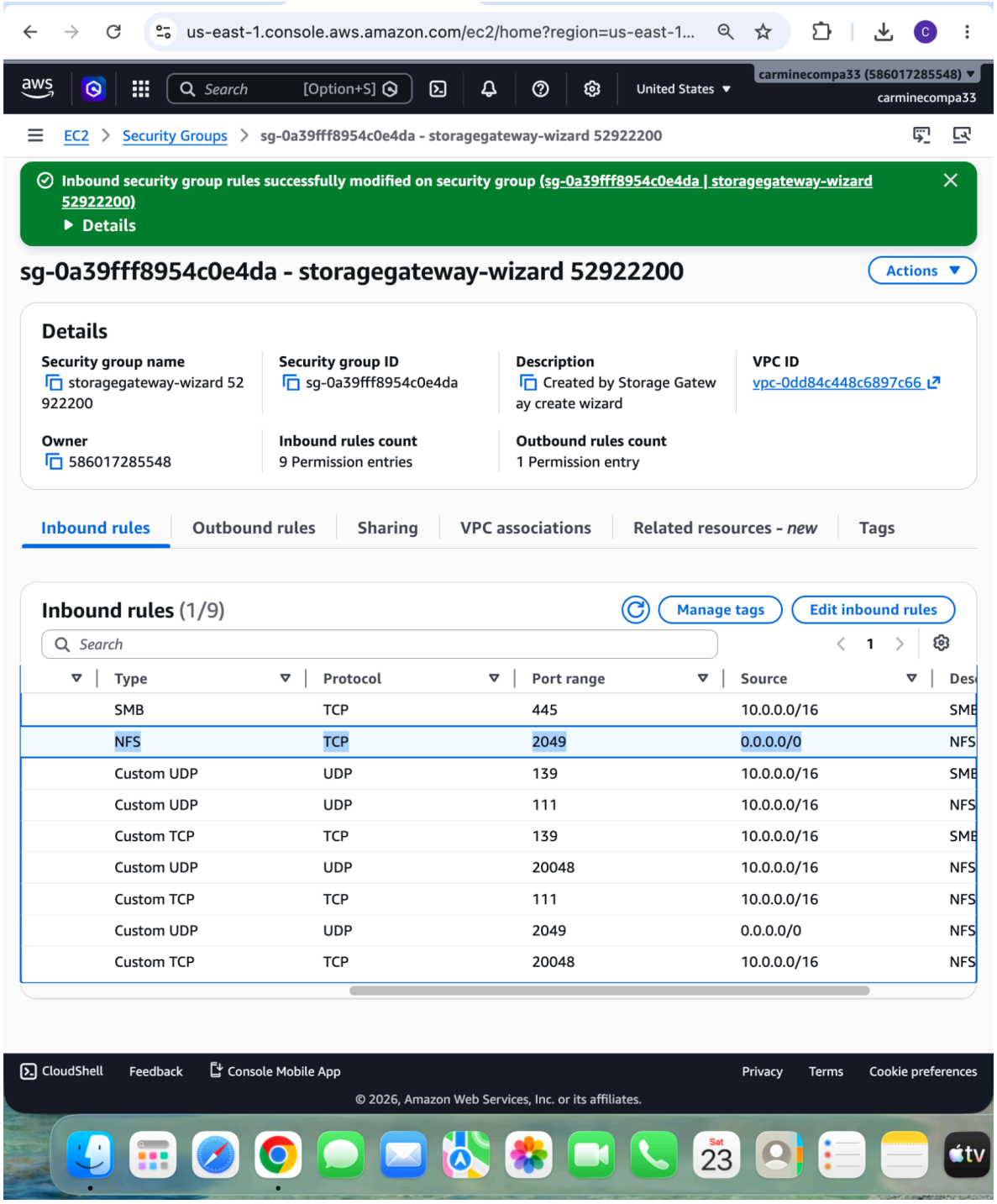


Create File Share. This allows me to build a bridge between my local computer and the storage.

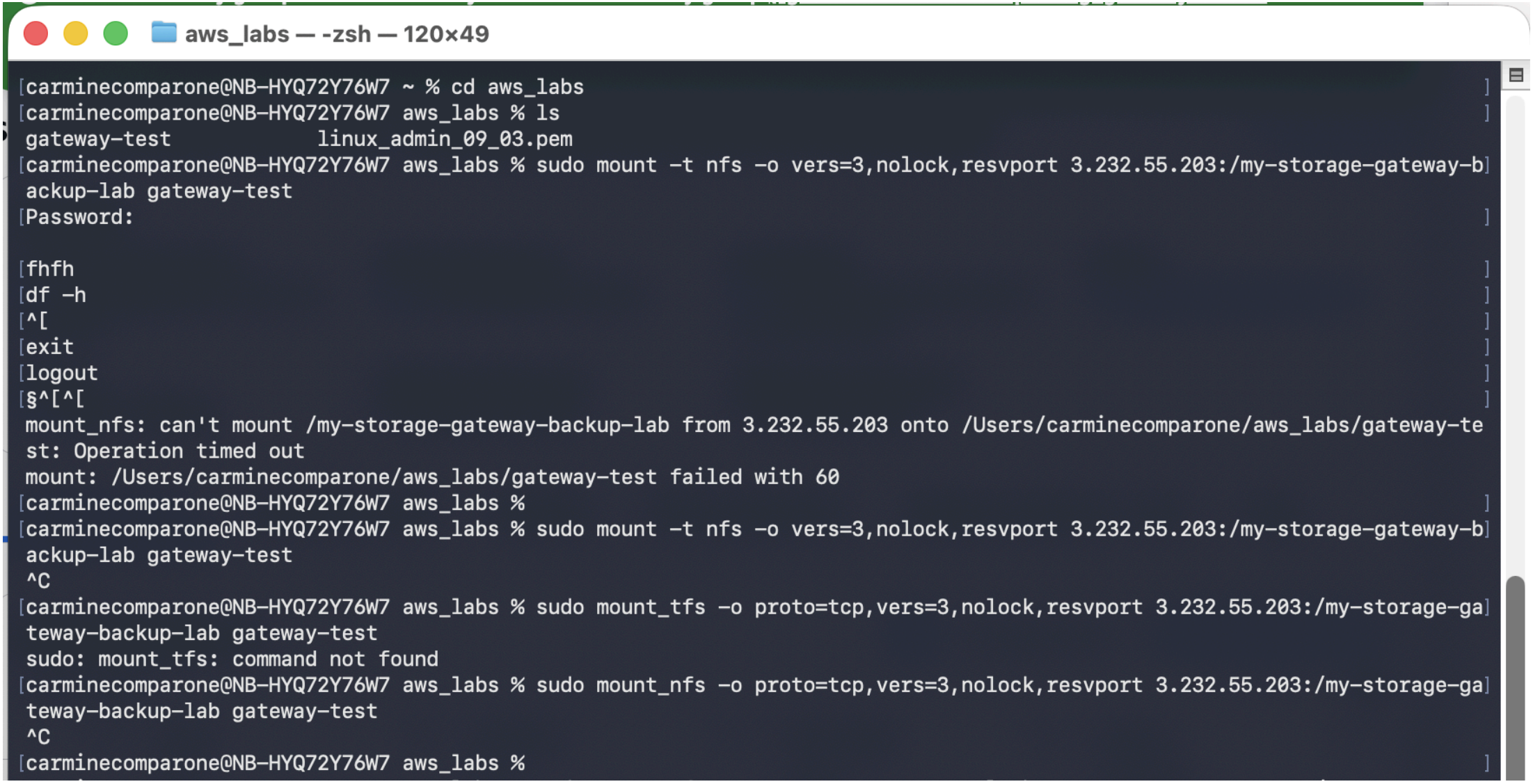
Without File Share, the storage gateway can’t expose the storage to the network.



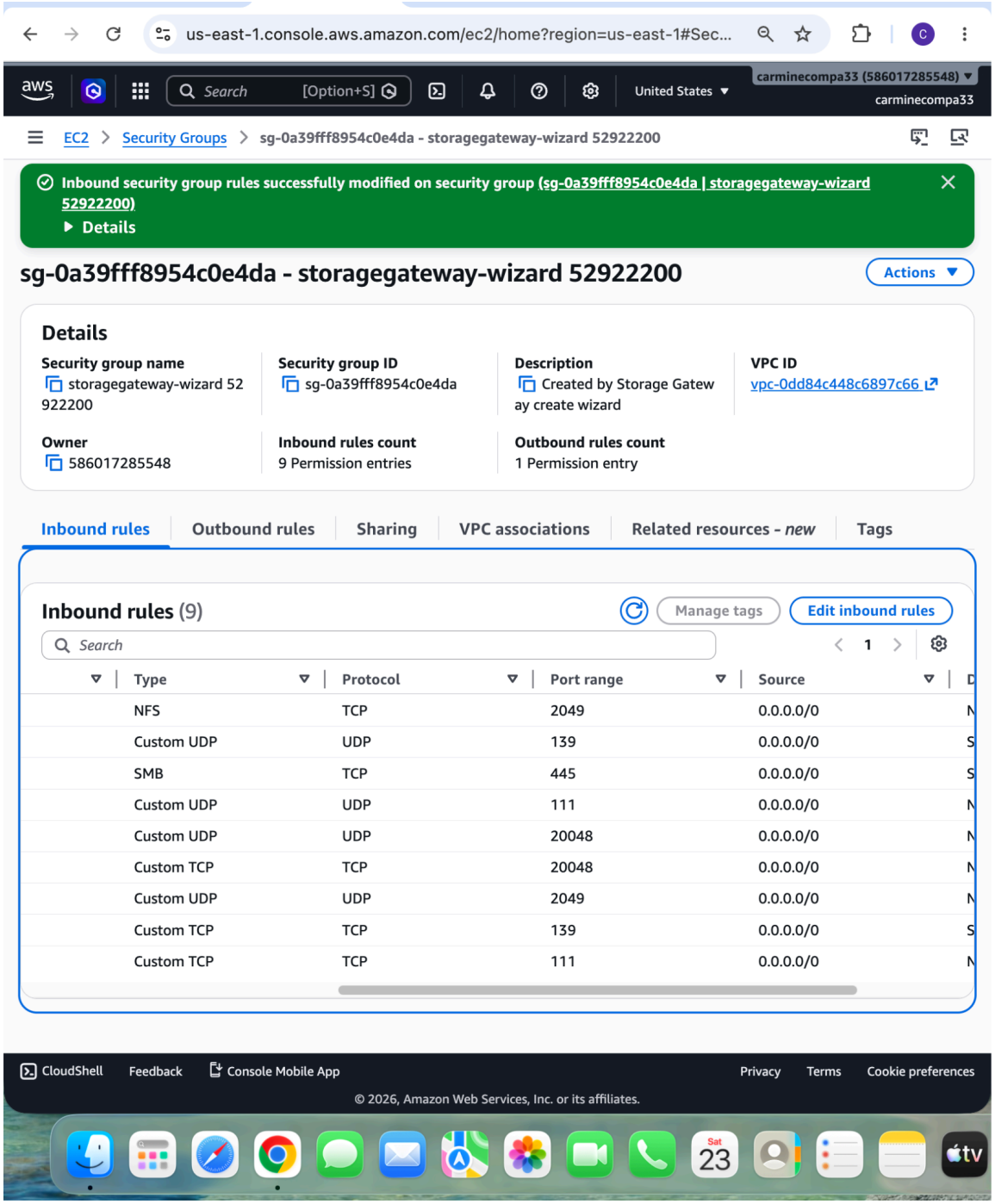
I make sure NFC TCP port is open to 0.0.0.0/0 on the EC2 machine



I try and I cannot mount the storage as NFS on my local laptop:

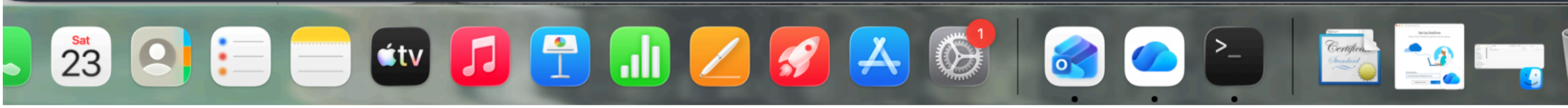


I open all the inbound rules of my EC2 machine



Now the mount works!

```
carminecomparone@NB-HYQ72Y76W7 aws_labs % sudo mount_nfs -o proto=tcp,vers=3,nolock,resvport 3.232.55.203:/my-storage-gateway-backup-lab gateway-test
carminecomparone@NB-HYQ72Y76W7 aws_labs % df -h
Filesystem                                    Size    Used   Avail Capacity iused ifree %iused  Mounted on
/dev/disk3s1s1                               460Gi    17Gi   347Gi     5%    453k  3.6G    0%   /
devfs                                        199Ki   199Ki     0Bi   100%     689     0  100%   /dev
/dev/disk3s6                                 460Gi    20Ki   347Gi     1%       0  3.6G    0%   /System/Volumes/VM
/dev/disk3s2                                 460Gi    15Gi   347Gi     5%    1.5k  3.6G    0%   /System/Volumes/Preboot
/dev/disk3s4                                 460Gi   775Mi   347Gi     1%     443  3.6G    0%   /System/Volumes/Update
/dev/disk1s2                                 500Mi   6.0Mi   482Mi     2%       1  4.9M    0%   /System/Volumes/xarts
/dev/disk1s1                                 500Mi   6.0Mi   482Mi     2%      45  4.9M    0%   /System/Volumes/iSCPreboot
/dev/disk1s3                                 500Mi   1.5Mi   482Mi     1%      96  4.9M    0%   /System/Volumes/Hardware
/dev/disk3s5                                 460Gi    78Gi   347Gi    19%    989k  3.6G    0%   /System/Volumes/Data
map auto_home                                  0Bi     0Bi     0Bi   100%       0     0     -   /System/Volumes/Data/home
/dev/disk3s1                                 460Gi    17Gi   347Gi     5%    458k  3.6G    0%   /System/Volumes/Update/mnt1
3.232.55.203:/my-storage-gateway-backup-lab  8.0Ei     0Bi   8.0Ei     0%       0  9.2E    0%   /Users/carminecomparone/aws_labs/gateway-test
carminecomparone@NB-HYQ72Y76W7 aws_labs % 44
```



After 5 minutes I can see the file in the S3 Bucket!

